



**VIT<sup>®</sup>**  
**B H O P A L**  
[www.vitbhopal.ac.in](http://www.vitbhopal.ac.in)

**School of Computing Science &  
Engineering & Artificial Intelligence(SCAI)**

**Project Report File**

**Computer Vision Based Parking Lot Occupancy  
Detection and Analysis System**

**Computer Vision  
(CSE3006) Slot:  
B14+D21  
Class no: BL2026270100300**

**Submitted By:**

Name: Raghav Khare  
Reg. no.: 24BAI10086

**Submitted to:**

Dr. RAJNEESH KUMAR PATEL  
Assistant Professor Grade 2, SCOPE

## 2. Introduction

### 2.1 Background

Parking-space availability is an important problem in modern parking facilities. A parking lot may contain a large number of individual spaces, making it difficult to determine the current occupancy status manually. A computer vision-based system can assist in analyzing parking-lot images and determining which parking spaces are empty or occupied.

This project develops a **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** using image processing and machine-learning techniques. The system processes parking-lot images together with parking-space annotations, extracts individual parking-space regions, and classifies each region as either **empty** or **occupied**.

The project uses a **PKLot-derived parking-space dataset** provided in COCO annotation format. The relevant annotations identify parking spaces as either space-empty or space-occupied.

### 2.2 Proposed Approach

The developed system follows a modular computer vision pipeline. First, parking-space regions are obtained from the provided bounding-box annotations. Each parking-space region is then preprocessed by resizing it to **64 × 64 pixels**, converting it to grayscale, and normalizing its pixel values.

After preprocessing, **Histogram of Oriented Gradients (HOG)** is used to extract visual features representing structural and edge information from each parking-space image. The resulting representation contains **1,764 features**.

These features are provided to an **RBF-kernel Support Vector Machine (SVM)** classifier. The classifier predicts whether each parking space is empty or occupied. Probability calibration is performed using CalibratedClassifierCV with sigmoid calibration and 5-fold cross-validation.

### 2.3 System Functionality

The system provides the following major functionality:

- Preparation of parking-space image crops from COCO annotations.
- Image preprocessing and normalization.
- HOG feature extraction.
- Training of an SVM-based occupancy classifier.
- Validation and test-set evaluation.
- Parking-space occupancy prediction.
- Calculation of total, occupied, and empty parking spaces.
- Calculation of overall occupancy percentage.
- Generation of an annotated parking-lot image.

- Generation of per-space and summary CSV reports.

For the included sample image, the system analyzes **100 parking spaces**, predicts **59 occupied spaces** and **41 empty spaces**, and calculates an overall occupancy of **59.00%**.

## 2.4 Dataset and Model Development

A balanced subset of the dataset was prepared for model development. The training set contains **6,000 samples**, consisting of 3,000 empty and 3,000 occupied parking-space images. The validation and test sets each contain **2,000 samples**, with equal numbers of empty and occupied samples.

The trained classifier achieved **96.50% accuracy and 96.54% F1-score on the final test set**, demonstrating that the developed approach can effectively distinguish between empty and occupied parking-space regions in the evaluated dataset.

## 2.5 Scope of the Project

The current implementation is designed as an **offline image-analysis system**. Parking-space locations are obtained from provided annotation information, while the machine-learning model performs the occupancy classification. The system does not currently perform automatic parking-space discovery from an unannotated image.

The project therefore focuses primarily on the **computer vision-based classification and analysis of predefined parking spaces**, along with quantitative reporting of parking occupancy.

## 2.6 Introduction Summary

In summary, this project demonstrates the application of computer vision and machine learning to parking-lot occupancy analysis. The combination of image preprocessing, HOG feature extraction, and calibrated SVM classification provides a lightweight approach for identifying parking-space occupancy. The modular implementation also separates dataset processing, preprocessing, classification, evaluation, analysis, and reporting, making the system easier to test and extend.

## 3. Problem Statement

### 3.1 Problem Overview

Parking facilities in shopping centers, universities, offices, hospitals, airports, and other public locations often contain a large number of individual parking spaces. Determining the availability of these spaces manually can be inefficient, especially when the parking area is large or frequently changes throughout the day.

The objective of this project is to develop a **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** that can automatically analyze a parking-lot image and determine the occupancy status of individual parking spaces.

The system takes a parking-lot image along with parking-space annotations as input. Each annotated parking space is extracted from the image and processed individually. A machine-learning classifier

then predicts whether the particular parking space is **empty** or **occupied**. The predictions are subsequently combined to generate an overall analysis of the parking lot.

The project follows the computer-vision workflow of image preprocessing, feature extraction, classification, prediction, and result visualization. The implementation uses **HOG (Histogram of Oriented Gradients)** features and an **SVM (Support Vector Machine)** classifier for parking-space occupancy classification. The project documentation specifies image preprocessing, HOG feature extraction, and SVM classification as the core methodology.

### 3.2 Existing Problem

In a conventional parking facility, information about available parking spaces may be obtained through:

- Manual observation by security or parking personnel.
- Drivers physically searching for an empty parking space.
- Basic parking sensors installed individually in spaces.
- Dedicated commercial parking-management systems.

For an image-based solution, the primary challenge is to correctly determine the occupancy state of each parking space from visual information.

A parking-lot image may contain:

- Multiple vehicles.
- Empty parking spaces.
- Different vehicle orientations.
- Shadows and illumination variations.
- Different parking-space sizes and positions.
- Background objects and road surfaces.

Therefore, simply examining the complete image is not sufficient. The system needs to identify individual parking spaces and analyze them separately.

### 3.3 Problem Definition

The problem addressed by this project can be formally stated as follows:

**Given an image of a parking lot and the locations of individual parking spaces, determine whether each parking space is empty or occupied and generate an overall occupancy analysis of the parking lot.**

For every parking space, the system performs the following operations:

1. Locate the parking space using its bounding-box annotation.

2. Extract the corresponding image region.
3. Resize and convert the extracted region into a suitable representation.
4. Extract HOG features from the processed image.
5. Classify the parking space using the trained SVM model.
6. Assign the predicted class as **empty** or **occupied**.
7. Calculate the confidence associated with the prediction.
8. Mark the parking space on the original image.
9. Generate a detailed CSV report.
10. Calculate the overall parking occupancy percentage.

The project therefore converts raw visual information into structured parking-occupancy information.

### 3.4 Input and Output

#### Input

The system requires two primary inputs:

##### 1. Parking-lot image

A JPG image containing the parking area to be analyzed.

##### 2. Parking-space annotations

A COCO-format JSON file containing the bounding boxes corresponding to individual parking spaces.

The dataset used by the project contains separate categories for empty and occupied parking spaces, while the project uses the parking-space bounding boxes to extract individual spaces for classification.

#### Output

The system produces three major outputs.

##### 1. Annotated Parking-Lot Image

The original parking-lot image is annotated with the predicted occupancy status of each detected parking space.

- **Green:** predicted empty
- **Red:** predicted occupied

This provides a visual representation of parking availability.

##### 2. Detailed CSV Report

A CSV file is generated containing information about individual parking spaces, including their predicted occupancy status and associated prediction information.

### 3. Parking Summary Report

A summary CSV contains aggregate information such as:

| Parameter            | Description                                |
|----------------------|--------------------------------------------|
| Total Spaces         | Total number of analyzed parking spaces    |
| Occupied Spaces      | Number of spaces predicted as occupied     |
| Empty Spaces         | Number of spaces predicted as empty        |
| Occupancy Percentage | Percentage of spaces predicted as occupied |

The README specifies that the system produces an annotated image and structured CSV outputs as part of the analysis workflow.

### 3.5 Major Challenges

The problem involves several technical challenges.

#### 3.5.1 Individual Parking-Space Extraction

The parking-lot image contains many spaces within a single image. Each space must be isolated before classification.

The system therefore uses the bounding-box information from the COCO annotations to crop individual parking-space regions.

#### 3.5.2 Image Preprocessing

Images can have different dimensions and visual characteristics. To provide a consistent input to the feature-extraction stage, each parking-space crop is resized to a fixed resolution and converted into grayscale before feature extraction.

The project uses a **64 × 64** image representation for HOG feature extraction.

#### 3.5.3 Feature Representation

A raw image contains a large number of pixel values and may not provide the most useful representation directly to a traditional machine-learning classifier.

The project therefore uses **Histogram of Oriented Gradients (HOG)** to represent the visual structure of each parking-space crop.

The resulting feature vector contains **1,764 features**, which are supplied to the classifier.

### 3.5.4 Empty vs. Occupied Classification

The main classification problem is binary:

$$Parking\ Space \rightarrow \begin{cases} Empty \\ Occupied \end{cases}$$

The project uses an **SVM-based classifier** to perform this classification. The model is trained using labeled parking-space samples and subsequently used to predict previously unseen samples.

### 3.5.5 Generating Reliable Overall Statistics

Individual predictions need to be converted into meaningful parking-lot statistics.

For example, if a parking lot contains 100 spaces and 59 are classified as occupied:

$$\begin{aligned} Occupancy\ Percentage &= \frac{Occupied\ Spaces}{Total\ Spaces} \times 100 \\ &= \frac{59}{100} \times 100 = 59\% \end{aligned}$$

Thus, the system provides both **space-level predictions** and **parking-lot-level statistics**.

## 3.6 Project Scope

The scope of the current implementation is deliberately defined around **image-based parking occupancy analysis**.

### Included in the scope

- Processing parking-lot images.
- Reading COCO-format annotations.
- Extracting individual parking spaces.
- Image preprocessing.
- HOG feature extraction.
- SVM-based classification.
- Empty/occupied prediction.
- Prediction confidence calculation.
- Annotated image generation.
- CSV report generation.
- Overall occupancy calculation.
- Model evaluation using validation and test data.
- Automated testing of important project components.

### Outside the current scope

The current implementation does not include:

- Live CCTV/video-stream processing.
- Automatic discovery of parking-space locations without annotations.
- A web-based dashboard.
- Mobile application integration.
- Database-backed parking management.
- Automatic license-plate recognition.
- Real-time parking reservations.

These are considered possible future enhancements rather than requirements of the current implementation.

### 3.7 Expected Solution

The expected solution is a modular Python application capable of taking a parking-lot image and its corresponding annotations and producing an automated occupancy analysis.

The complete processing pipeline is:

**Parking Image + Annotations**

↓

**Parking-Space Extraction**

↓

**Image Preprocessing**

↓

**HOG Feature Extraction**

↓

**Trained SVM Classifier**

↓

**Empty / Occupied Prediction**

↓

**Confidence Calculation**

↓

**Annotated Image + CSV Reports + Occupancy Statistics**

This approach provides a structured solution to the problem while demonstrating the practical application of computer-vision and machine-learning concepts.

### 3.8 Problem Statement Summary

The project addresses the problem of **automatically determining parking-space occupancy from parking-lot images**. The system analyzes individual parking spaces, extracts visual features using



HOG, classifies each space using an SVM model, and combines the predictions into a complete parking-lot occupancy report.

The solution is designed as a modular computer-vision application rather than a simple image-classification script, with separate components for dataset processing, preprocessing, classification, evaluation, analysis, and reporting. This modular approach also aligns with the project's requirement for a structured implementation containing multiple functional modules.

#### **4. Functional Requirements**

Functional requirements define the specific operations that the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** must perform. They describe what the system should do from the time the input data is provided until the final parking-occupancy results are generated.

The project is designed as a modular system with separate components for dataset processing, image preprocessing, feature extraction, classification, evaluation, analysis, and reporting.

##### **4.1 FR-01: Dataset Loading and Annotation Processing**

The system shall be able to read the parking-lot dataset and its corresponding COCO-format annotation files.

The dataset-processing module shall:

- Read the annotation JSON file.
- Identify the available image files.
- Read parking-space bounding-box annotations.
- Identify the relevant parking-space categories.
- Associate each annotation with its corresponding image.
- Extract the bounding-box coordinates required for cropping.

The system shall ignore irrelevant annotation categories and process the categories required for parking-space occupancy analysis.

##### **4.2 FR-02: Parking-Space Extraction**

The system shall extract individual parking-space regions from a parking-lot image using the bounding-box coordinates provided in the annotations.

For every valid parking-space annotation, the system shall:

1. Read the bounding-box coordinates.
2. Locate the corresponding region in the original image.
3. Crop the parking-space region.
4. Validate that the resulting crop is usable.

5. Pass the crop to the preprocessing module.

This allows the classifier to analyze individual parking spaces rather than the complete parking-lot image.

#### 4.3 FR-03: Image Preprocessing

The system shall preprocess each extracted parking-space image before feature extraction.

The preprocessing module shall:

- Load the image.
- Resize it to **64 × 64 pixels**.
- Convert the image to grayscale.
- Normalize the pixel values.
- Return the processed image in a consistent format.

Standardizing the input ensures that the feature-extraction and classification stages receive consistent data.

#### 4.4 FR-04: HOG Feature Extraction

The system shall extract visual features from every preprocessed parking-space image using **Histogram of Oriented Gradients (HOG)**.

The HOG feature-extraction module shall use the configured parameters for:

- Number of orientations.
- Pixels per cell.
- Cells per block.
- Block normalization.

The extracted HOG representation shall be converted into a feature vector suitable for machine-learning classification.

The implemented configuration produces a **1,764-dimensional feature vector** for each 64 × 64 parking-space image.

#### 4.5 FR-05: Model Training

The system shall provide functionality for training the parking-space occupancy classifier.

The training process shall:

1. Load the prepared training dataset.
2. Extract HOG features from the training samples.

3. Assign the corresponding class labels.
4. Train the SVM classifier.
5. Calibrate the classifier for probability/confidence estimation.
6. Save the trained model to a model file.

The classifier shall use two classes:

**Class ID Class**

- |   |          |
|---|----------|
| 0 | Empty    |
| 1 | Occupied |

The project uses an SVM-based classification approach for distinguishing empty and occupied parking spaces.

#### **4.6 FR-06: Parking-Space Classification**

The system shall classify each extracted parking-space image as either:

- **Empty**, or
- **Occupied**.

For each parking-space crop, the system shall:

1. Apply the same preprocessing used during training.
2. Extract HOG features.
3. Pass the feature vector to the trained classifier.
4. Obtain the predicted class.
5. Convert the numerical prediction into a human-readable label.

The prediction process shall use the trained model without requiring the model to be retrained for every analysis operation.

#### **4.7 FR-07: Prediction Confidence**

The system shall calculate a confidence/probability value associated with each parking-space prediction when supported by the trained classifier.

The confidence value shall represent the model's estimated likelihood for the predicted class.

This information can be used to understand how confidently the model classified each individual parking space.

#### **4.8 FR-08: Parking-Lot Analysis**

The system shall combine the individual parking-space predictions to produce an overall parking-lot analysis.

The analysis module shall calculate:

- Total number of parking spaces.
- Number of occupied spaces.
- Number of empty spaces.
- Overall occupancy percentage.

The occupancy percentage shall be calculated using:

$$\text{Occupancy Percentage} = \frac{\text{Occupied Spaces}}{\text{Total Spaces}} \times 100$$

For example, if 59 out of 100 spaces are predicted to be occupied:

$$\text{Occupancy Percentage} = \frac{59}{100} \times 100 = 59\%$$

#### **4.9 FR-09: Annotated Image Generation**

The system shall generate an annotated version of the input parking-lot image.

Each analyzed parking space shall be visually marked according to its predicted occupancy status.

The implemented visualization uses:

- **Green bounding box → Empty**
- **Red bounding box → Occupied**

The annotations shall be drawn over the original parking-lot image while preserving the underlying image information.

This allows a user to visually verify the predictions produced by the system.

#### **4.10 FR-10: Detailed CSV Report Generation**

The system shall generate a CSV report containing the analysis results for individual parking spaces.

The report shall contain information corresponding to the analyzed parking spaces, including their predicted occupancy status and relevant prediction information.

The CSV format allows the results to be easily opened and further processed using spreadsheet or data-analysis software.

#### **4.11 FR-11: Summary Report Generation**

The system shall generate a separate summary report containing aggregate parking-lot statistics.

The summary report shall include:

| Field                | Description                     |
|----------------------|---------------------------------|
| total_spaces         | Total number of analyzed spaces |
| occupied_spaces      | Number of occupied spaces       |
| empty_spaces         | Number of empty spaces          |
| occupancy_percentage | Overall occupancy percentage    |

The project documentation specifies structured CSV output as part of the final analysis workflow.

#### **4.12 FR-12: Model Evaluation**

The system shall provide functionality for evaluating the trained classifier using validation and test datasets.

The evaluation module shall calculate classification-performance metrics including:

- Accuracy
- Precision
- Recall
- F1-score

It shall also generate confusion matrices to visualize classification results.

The project documentation defines accuracy, precision, recall, F1-score, and confusion matrices as evaluation measures for the classifier.

#### **4.13 FR-13: Automated Testing**

The system shall include automated tests for important project components.

The test suite shall verify functionality such as:

- Bounding-box cropping.
- Image preprocessing.
- HOG feature extraction.
- Sample annotation availability.
- Sample image availability.
- Output image generation.
- CSV report generation.

The project includes a dedicated test suite to validate the implemented functionality.

#### 4.14 FR-14: Command-Line Execution

The system shall provide a command-line interface through which a user can perform parking-lot analysis.

The application shall accept parameters such as:

- Input parking-lot image.
- Annotation JSON file.
- Output image location.
- CSV output location.

This allows the system to be executed without requiring a graphical user interface or web application.

#### 4.15 Functional Requirements Summary

| ID    | Functional Requirement          | Main Module              |
|-------|---------------------------------|--------------------------|
| FR-01 | Load dataset and annotations    | dataset.py               |
| FR-02 | Extract parking-space crops     | dataset.py / analyzer.py |
| FR-03 | Preprocess images               | preprocessing.py         |
| FR-04 | Extract HOG features            | preprocessing.py         |
| FR-05 | Train SVM classifier            | classifier.py            |
| FR-06 | Classify parking spaces         | classifier.py            |
| FR-07 | Calculate prediction confidence | classifier.py            |
| FR-08 | Calculate parking statistics    | analyzer.py              |
| FR-09 | Generate annotated image        | analyzer.py              |
| FR-10 | Generate detailed CSV           | reporter.py              |
| FR-11 | Generate summary CSV            | reporter.py              |
| FR-12 | Evaluate trained model          | evaluator.py             |
| FR-13 | Run automated tests             | tests/                   |
| FR-14 | Execute system through CLI      | main.py                  |

These requirements establish the complete functional scope of the proposed system, from raw parking-lot data preparation to final prediction, visualization, reporting, and evaluation.

## 5. Non-Functional Requirements

Non-functional requirements define the **quality attributes and operational constraints** of the Computer Vision Based Parking Lot Occupancy Detection and Analysis System. Unlike functional requirements, which describe what the system does, non-functional requirements describe **how effectively, reliably, maintainably, and securely the system should perform those functions**.

The project requirements specify that the system should include at least four non-functional requirements in addition to its functional modules.

### 5.1 NFR-01: Performance

The system should process parking-space images efficiently without unnecessary computational overhead.

The system shall:

- Resize images to a fixed resolution before feature extraction.
- Extract a fixed-size HOG feature vector for each parking-space crop.
- Avoid repeated preprocessing of the same image.
- Process parking spaces independently so that analysis can proceed systematically.
- Store the trained classifier so that the model does not need to be retrained for every prediction.

The use of HOG features and an SVM classifier provides a relatively lightweight machine-learning pipeline compared with more computationally intensive deep-learning approaches.

### 5.2 NFR-02: Accuracy and Reliability

The system should provide reliable predictions of parking-space occupancy.

Reliability shall be supported through:

- A labeled dataset containing empty and occupied parking-space samples.
- Separate training, validation, and test datasets.
- Consistent preprocessing between training and prediction.
- Model evaluation using accuracy, precision, recall, and F1-score.
- Confusion-matrix analysis.
- Automated testing of important system components.

The final model evaluation reported by the project achieved **96.50% test accuracy** and an **F1-score of 96.54%**, demonstrating that the implemented classifier performs effectively on the selected test subset.

### 5.3 NFR-03: Maintainability

The system should be easy to understand, modify, debug, and extend.

To achieve this, the implementation is divided into separate modules based on their responsibilities.

For example:

- `dataset.py` handles dataset and annotation processing.
- `preprocessing.py` handles image preprocessing and HOG extraction.
- `classifier.py` handles model training and prediction.
- `evaluator.py` handles model evaluation.
- `analyzer.py` handles parking-lot analysis and visualization.
- `reporter.py` handles report generation.
- `main.py` provides the application entry point.

This modular structure prevents all functionality from being placed inside a single script and makes individual components easier to modify or replace.

The project requirements also emphasize modular and clean implementation with proper project structure.

### 5.4 NFR-04: Scalability

The system should be capable of handling parking lots containing different numbers of parking spaces.

The analysis pipeline operates on individual parking-space annotations rather than assuming a fixed number of spaces.

Therefore, the same workflow can be applied to an image containing:

- A small parking area.
- A medium-sized parking lot.
- A larger parking area with many annotated spaces.

The number of spaces is determined from the input annotation file, allowing the analysis to adapt to the supplied parking-lot image.

### 5.5 NFR-05: Usability

The system should be straightforward for a technical user to execute and understand.

The application is operated through a command-line interface and accepts clearly defined input and output parameters.

The user should be able to specify:



- Input parking-lot image.
- Parking-space annotation file.
- Annotated-image output path.
- CSV report output path.

The system should also display a concise summary after analysis, including the total number of spaces, occupied spaces, empty spaces, and occupancy percentage.

The generated annotated image provides an additional visual representation that makes the classification results easier to interpret.

## **5.6 NFR-06: Robustness and Error Handling**

The system should handle invalid or unexpected inputs without causing uncontrolled program failures wherever practical.

The implementation should validate conditions such as:

- Missing image files.
- Missing annotation files.
- Invalid image data.
- Invalid bounding-box information.
- Empty datasets.
- Mismatched feature and label counts.
- Invalid or unusable image crops.

Where an error is detected, the system should provide a meaningful error message rather than silently producing an incorrect result.

This is particularly important because incorrect input data can otherwise result in invalid computer-vision predictions.

## **5.7 NFR-07: Reproducibility**

The system should produce reproducible training and evaluation results when executed using the same dataset and configuration.

The project uses a fixed random-state configuration for operations involving randomized dataset selection and model training.

Important configuration parameters such as:

- Image size.
- Maximum number of samples.

- Test size.
- Random state.
- Model configuration.

are centralized in `config.py`.

This reduces the possibility of different modules using inconsistent configuration values.

### **5.8 NFR-08: Testability**

The system should be structured so that its individual components can be tested independently.

Automated tests are provided for important operations including:

- Image preprocessing.
- HOG feature extraction.
- Bounding-box cropping.
- Sample image availability.
- Sample annotation availability.
- Output image generation.
- CSV report generation.

The project test suite successfully verifies these core operations. The project documentation also includes automated testing as part of the implementation workflow.

### **5.9 NFR-09: Documentation**

The project shall maintain sufficient documentation to allow another user or developer to understand and reproduce the system.

Documentation shall include:

- Project overview.
- Problem statement.
- Objectives.
- Functional requirements.
- Dataset information.
- Model methodology.
- Installation instructions.
- Training instructions.
- Evaluation procedure.

- Application execution instructions.
- Project structure.
- Limitations.
- Future enhancements.

The project also maintains README.md and statement.md as part of the GitHub project documentation.

### **5.10 NFR-10: Version Control**

The project should maintain its source code using Git version control.

Version control should allow:

- Tracking changes to source code.
- Maintaining project history.
- Organizing development versions.
- Identifying modifications.
- Publishing the source code in a GitHub repository.

The project requirements specifically include Git/GitHub usage, repository organization, and documentation.

### **5.11 NFR-11: Portability**

The application should be executable on systems capable of running Python and the required project dependencies.

The project uses commonly available Python libraries including:

- OpenCV
- NumPy
- Pandas
- Scikit-learn
- Scikit-image
- Matplotlib
- Joblib
- Pytest

Dependencies are listed in requirements.txt, allowing the development environment to be recreated without manually installing each package individually.

### 5.12 NFR-12: Security and Data Integrity

The system should preserve the integrity of input data during analysis.

The analysis process should:

- Read the original image without modifying it.
- Read annotations without altering the original dataset.
- Generate analysis results as separate output files.
- Keep generated model and processed-data files separate from the original dataset.

This ensures that the source dataset remains unchanged during preprocessing, training, and analysis.

### 5.13 Non-Functional Requirements Summary

| ID     | Requirement            | Description                                                  |
|--------|------------------------|--------------------------------------------------------------|
| NFR-01 | Performance            | Process images and parking spaces efficiently.               |
| NFR-02 | Accuracy & Reliability | Produce and evaluate reliable occupancy predictions.         |
| NFR-03 | Maintainability        | Use modular, understandable, and maintainable code.          |
| NFR-04 | Scalability            | Support parking lots with different numbers of spaces.       |
| NFR-05 | Usability              | Provide simple CLI execution and understandable outputs.     |
| NFR-06 | Robustness             | Handle invalid inputs and processing errors appropriately.   |
| NFR-07 | Reproducibility        | Produce consistent results using fixed configuration.        |
| NFR-08 | Testability            | Allow important modules to be tested independently.          |
| NFR-09 | Documentation          | Provide sufficient technical and user documentation.         |
| NFR-10 | Version Control        | Maintain the project using Git/GitHub.                       |
| NFR-11 | Portability            | Run using a standard Python environment and dependencies.    |
| NFR-12 | Data Integrity         | Preserve original input data and separate generated outputs. |

### 5.14 Summary

The non-functional requirements ensure that the project is not only capable of producing parking-occupancy predictions but is also **reliable, maintainable, testable, reproducible, scalable, and properly documented**.

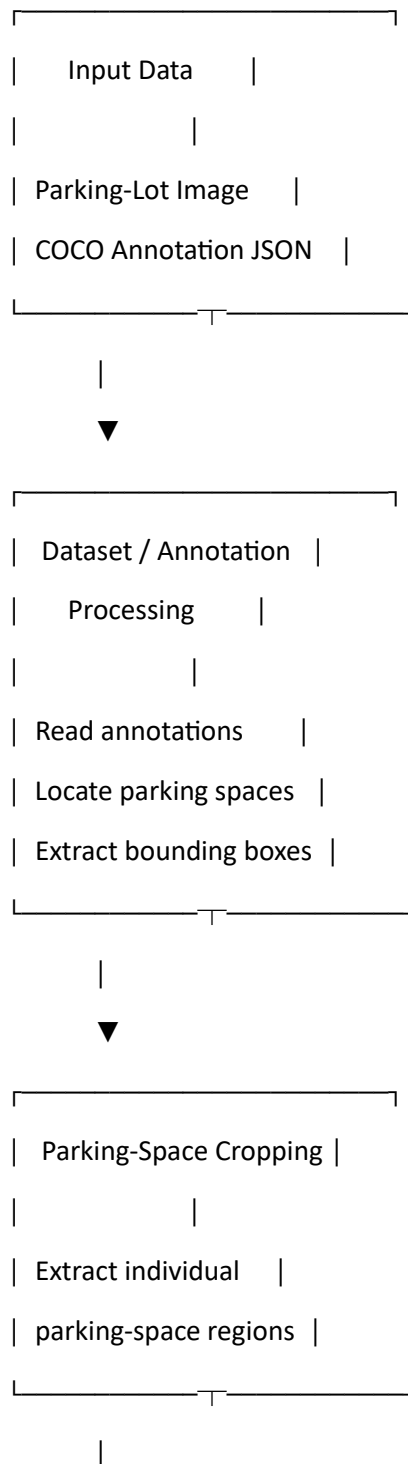
Together with the functional requirements, these requirements define the quality and implementation expectations of the complete system.

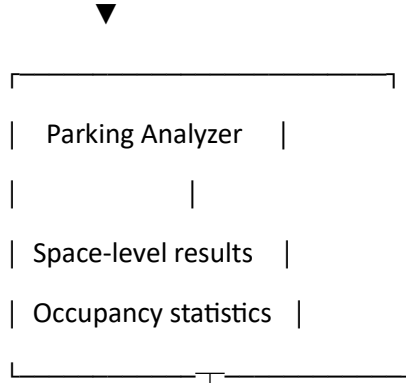
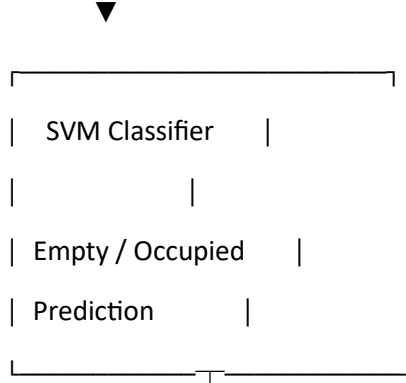
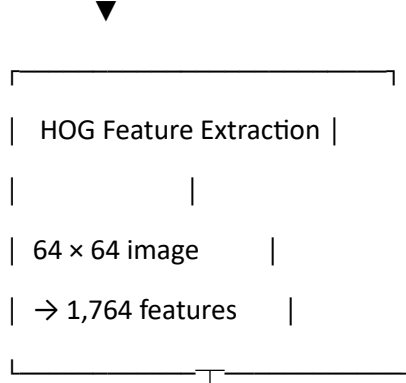
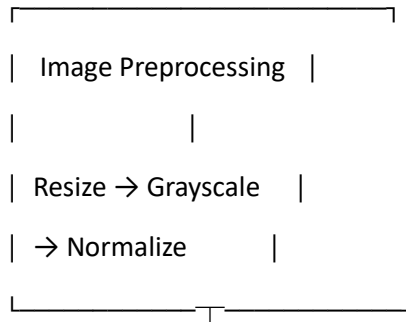
## 6. System Architecture

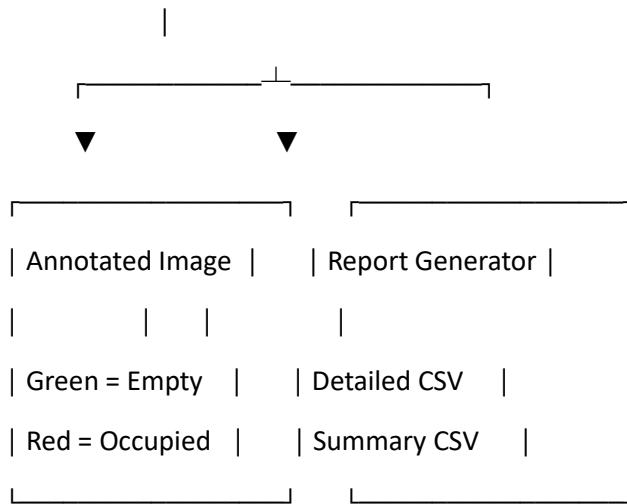
### 6.1 Architecture Overview

The **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** follows a modular, sequential processing architecture. The system is divided into independent components, where each component performs a specific responsibility and passes its output to the next stage.

The overall architecture can be represented as:







The architecture separates **data preparation, computer-vision processing, machine-learning classification, analysis, and reporting** into distinct stages. This modular organization is consistent with the project's requirement for a structured and maintainable implementation.

## 6.2 Architectural Components

The major components of the system are described below.

### 6.2.1 Input Layer

The input layer receives the information required to perform parking-space analysis.

It consists of:

- Parking-lot image.
- COCO-format annotation JSON file.

The image contains the visual representation of the parking lot, while the annotation file provides the locations of individual parking spaces.

### 6.2.2 Dataset and Annotation Processing Module

**Implementation:** `src/dataset.py`

This module is responsible for preparing the dataset used during model development.

Its responsibilities include:

- Reading COCO-format annotations.
- Mapping images to annotations.
- Identifying empty and occupied classes.
- Extracting bounding boxes.
- Creating individual parking-space crops.

- Organizing processed samples into training, validation, and test directories.
- Generating metadata describing the processed dataset.

The project uses a balanced subset of parking-space samples for model development.

### 6.2.3 Preprocessing Module

**Implementation:** `src/preprocessing.py`

The preprocessing module converts raw parking-space crops into a standardized representation.

The processing pipeline is:

Original Crop

|



Resize to 64 × 64

|



Convert to Grayscale

|



Normalize Pixel Values

|



Preprocessed Image

Standardizing the images ensures that the subsequent HOG feature-extraction stage receives consistent input.

### 6.2.4 Feature Extraction Module

The project uses **Histogram of Oriented Gradients (HOG)** for extracting visual information from each parking-space image.

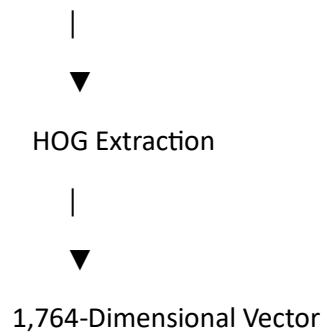
HOG captures local gradient and edge-orientation information, producing a numerical representation of the parking-space appearance.

The configured HOG pipeline generates a **1,764-dimensional feature vector** from each processed 64 × 64 image.

The output of this component is therefore:

Preprocessed Image





### 6.2.5 Classification Module

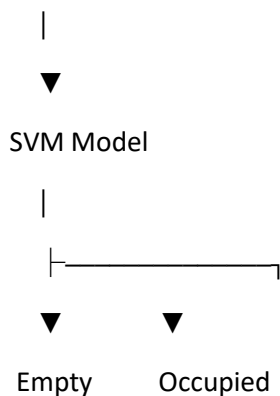
**Implementation:** `src/classifier.py`

The classification module contains the machine-learning model responsible for determining the occupancy state.

The project uses a calibrated **Support Vector Machine (SVM)** classifier.

The classifier receives the HOG feature vector and produces:

HOG Features



The model uses two classes:

- 0 → Empty
- 1 → Occupied

The project documentation describes SVM as the selected classification method for distinguishing empty and occupied parking spaces.

### 6.2.6 Evaluation Module

**Implementation:** `src/evaluator.py`

The evaluation module is responsible for measuring the performance of the trained classifier.

It evaluates the model using separate validation and test datasets.

The module calculates:

- Accuracy
- Precision
- Recall
- F1-score

It also generates confusion matrices to show the number of correctly and incorrectly classified samples.

The project uses these metrics as part of its model evaluation methodology.

### 6.2.7 Parking Analysis Module

**Implementation:** `src/analyzer.py`

This is the main computer-vision analysis component.

The analyzer receives:

- Parking-lot image.
- Parking-space annotations.
- Trained classifier.

For each parking space, it:

1. Finds the corresponding annotation.
2. Extracts the bounding-box region.
3. Preprocesses the crop.
4. Extracts HOG features.
5. Obtains the classifier prediction.
6. Determines the prediction confidence.
7. Records the result.
8. Draws the prediction on the original image.

After processing all spaces, it calculates the overall parking-lot statistics.

### 6.2.8 Reporting Module

**Implementation:** `src/reporter.py`

The reporting module converts the analysis results into structured output files.

It generates:

**Detailed CSV**

Contains space-level prediction information.

### **Summary CSV**

Contains aggregate information:

Total Spaces

Occupied Spaces

Empty Spaces

Occupancy Percentage

Separating reporting from analysis ensures that the core prediction logic does not depend on a particular output format.

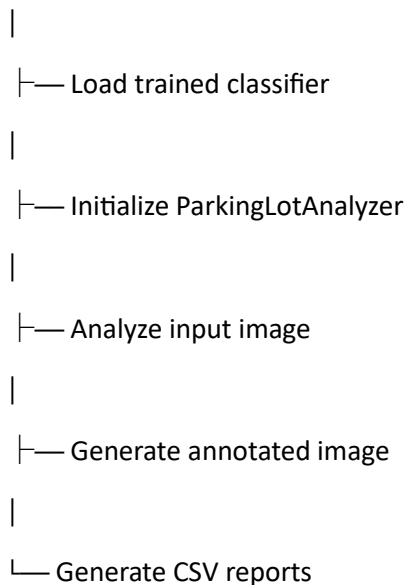
## **6.2.9 Application Entry Point**

**Implementation:** main.py

main.py acts as the main entry point of the application.

It connects the major components together:

main.py

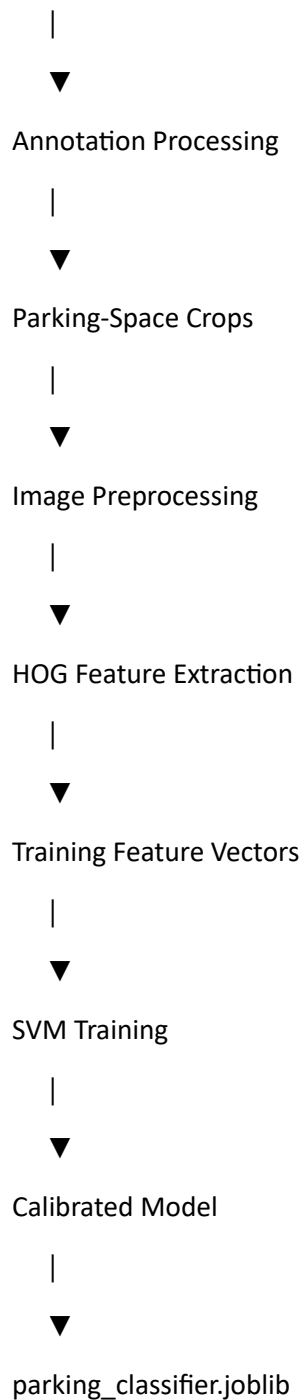


The user interacts with the system through command-line arguments rather than directly calling individual modules.

## **6.3 Training Architecture**

The training process is separate from the actual parking-lot analysis process.

COCO Dataset



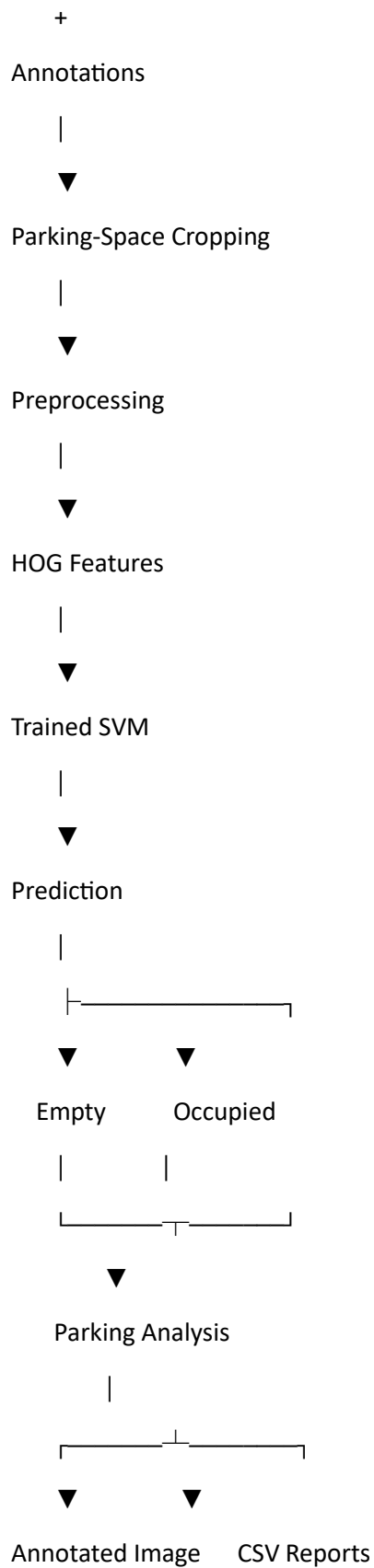
The saved model can subsequently be loaded by the application without repeating the training process.

This separation improves efficiency because model training is performed independently from normal prediction.

## 6.4 Prediction Architecture

During normal application execution, the trained model is loaded and used for inference.

Parking-Lot Image



This architecture allows the same trained model to be reused for multiple compatible parking-lot images.

## 6.5 Data Flow

The complete data flow can be summarized as follows:

| Stage              | Input                        | Processing                       | Output                     |
|--------------------|------------------------------|----------------------------------|----------------------------|
| Dataset Processing | COCO dataset                 | Read annotations and crop spaces | Parking-space samples      |
| Preprocessing      | Image crop                   | Resize, grayscale, normalize     | Standardized image         |
| Feature Extraction | Processed image              | HOG                              | 1,764-D vector             |
| Classification     | HOG vector                   | SVM prediction                   | Empty/Occupied             |
| Analysis           | Individual predictions       | Aggregate results                | Parking statistics         |
| Visualization      | Predictions + original image | Draw bounding boxes              | Annotated image            |
| Reporting          | Analysis results             | Convert to CSV                   | Detailed + summary reports |
| Evaluation         | Test/validation features     | Calculate metrics                | Metrics + confusion matrix |

## 6.6 Architectural Design Principles

The system architecture follows several important design principles.

### Separation of Responsibilities

Each module has a specific responsibility. For example, preprocessing does not perform classification, and reporting does not perform image processing.

### Modularity

Components can be modified or replaced independently. For example, the HOG feature extractor could eventually be replaced by another feature-extraction technique without completely redesigning the reporting module.

### Reusability

The trained classifier can be loaded and reused for multiple analysis operations.

### Testability

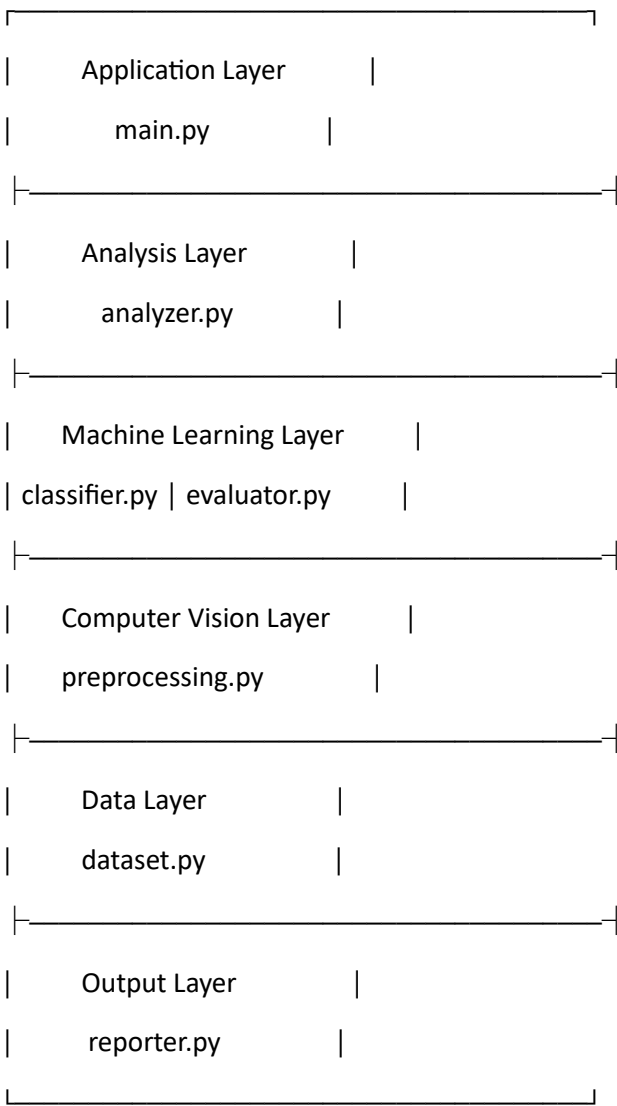
Individual operations can be tested independently, making it easier to identify errors in specific components.

### Extensibility

The architecture allows future additions such as automatic parking-space detection, deep-learning models, video processing, or a graphical interface without requiring the entire system to be rewritten.

6.7 Project Architecture Summary

The proposed system follows a **layered modular architecture**:



This architecture provides a clear separation between the **data layer, computer-vision processing, machine-learning layer, analysis layer, and application/output layer**. It also provides a suitable foundation for the design diagrams and implementation details presented in the following sections.

7. Design Diagrams

Design diagrams provide a visual representation of how the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** is structured and how its components interact.

The project requires appropriate design artifacts such as UML diagrams and workflow representation to document the system architecture and implementation.

This section contains:

1. Use Case Diagram
2. Workflow Diagram
3. Sequence Diagram
4. Class/Component Diagram
5. ER Diagram — not applicable because the current system does not use a database

## 7.1 Use Case Diagram

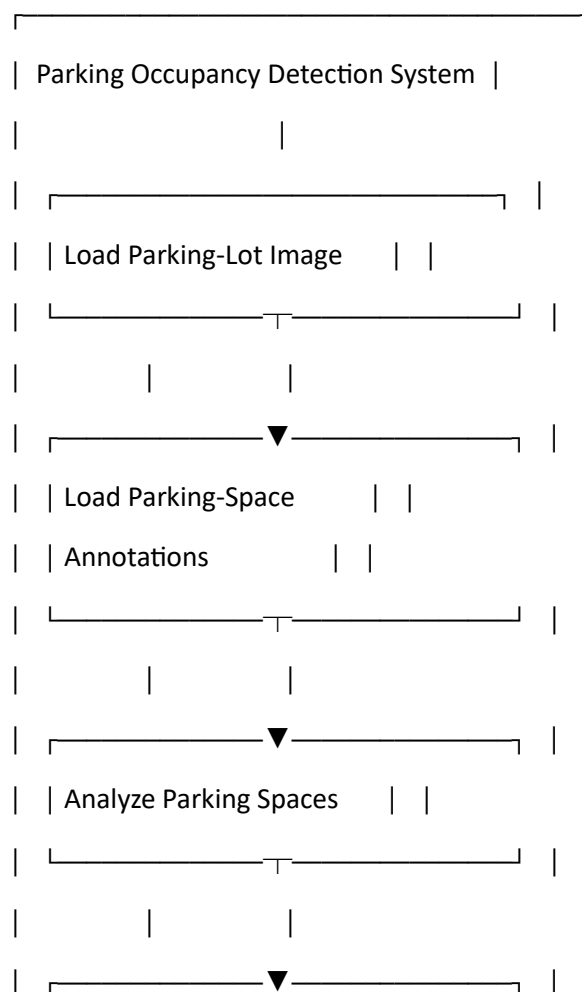
### 7.1.1 Purpose

The Use Case Diagram describes the interaction between the **User** and the parking occupancy analysis system.

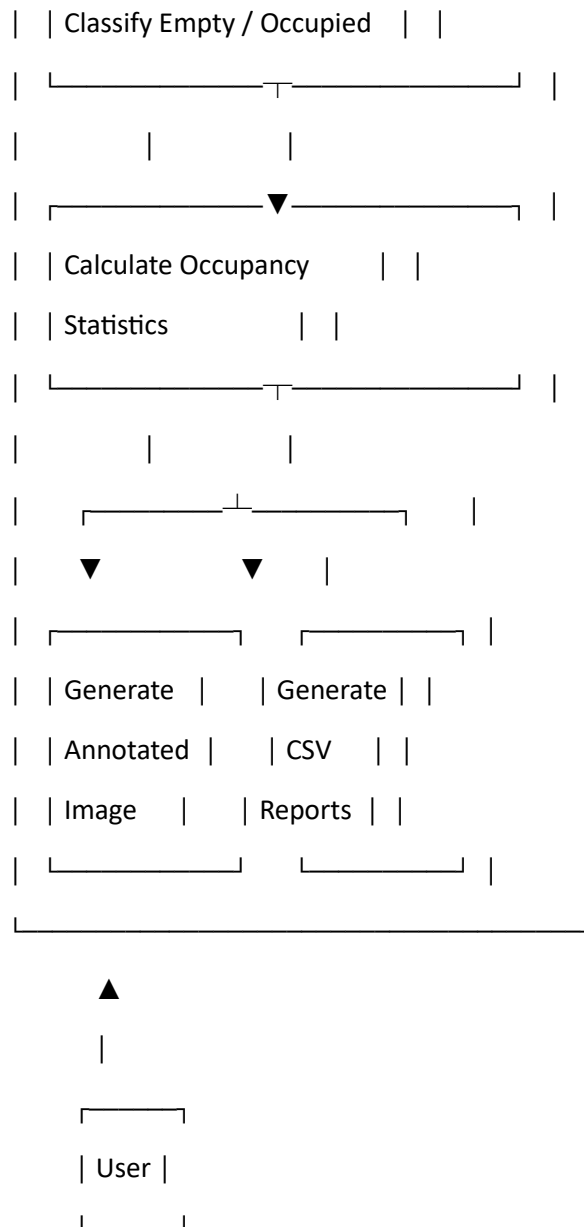
The system has one primary actor: **User**.

The user provides the required input files, executes the application, and receives the generated parking analysis results.

### 7.1.2 Use Case Representation







### 7.1.3 Use Case Descriptions

| Use Case                 | Actor  | Description                                                  |
|--------------------------|--------|--------------------------------------------------------------|
| Load Image               | User   | Provides the parking-lot image to be analyzed.               |
| Load Annotations         | User   | Provides the COCO-format annotation file.                    |
| Analyze Parking Spaces   | User   | Starts the occupancy-analysis process.                       |
| Classify Spaces          | System | Extracts features and predicts empty/occupied status.        |
| Calculate Statistics     | System | Calculates total, occupied, empty, and occupancy percentage. |
| Generate Annotated Image | System | Marks individual spaces according to predictions.            |

| Use Case         | Actor  | Description                             |
|------------------|--------|-----------------------------------------|
| Generate Reports | System | Creates detailed and summary CSV files. |

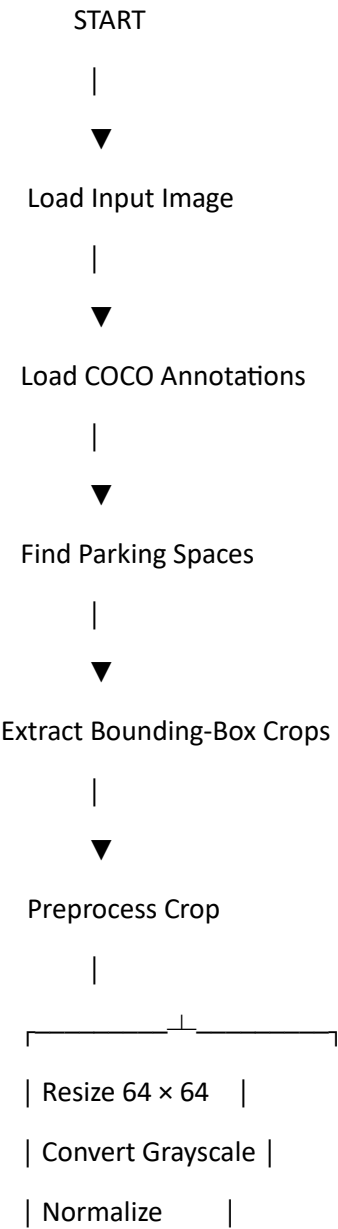
7.2 Workflow Diagram

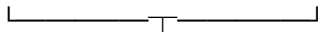
7.2.1 Purpose

The workflow diagram represents the complete processing pipeline from input acquisition to final output.

The system follows a sequential workflow in which each stage produces the information required by the next stage.

7.2.2 Workflow





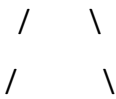
Extract HOG Features



Apply Trained SVM



Empty / Occupied?

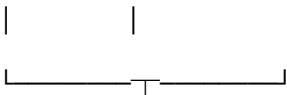


Empty

Occupied



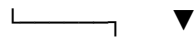
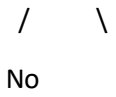
Green Annotation    Red Annotation



Process Next Space



All Spaces Done?

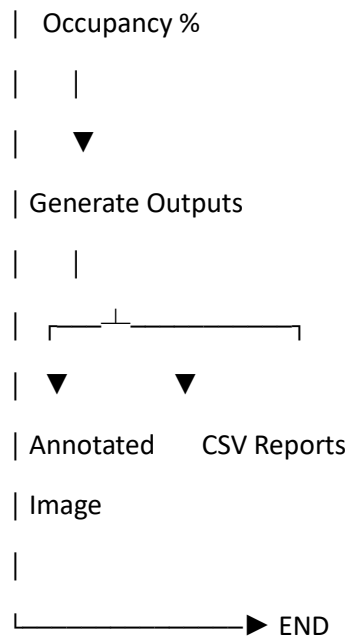


| Calculate:

| Total Spaces

| Occupied Spaces

| Empty Spaces



### Workflow Explanation

The process begins when the user supplies the parking-lot image and annotation file.

The system identifies the parking-space bounding boxes and extracts the corresponding image regions. Each crop is then resized, converted to grayscale, and normalized before HOG features are extracted.

The resulting feature vector is passed to the trained SVM classifier. The classifier predicts whether the space is empty or occupied.

The process continues until all parking spaces have been analyzed. The individual predictions are then aggregated to calculate the overall occupancy statistics.

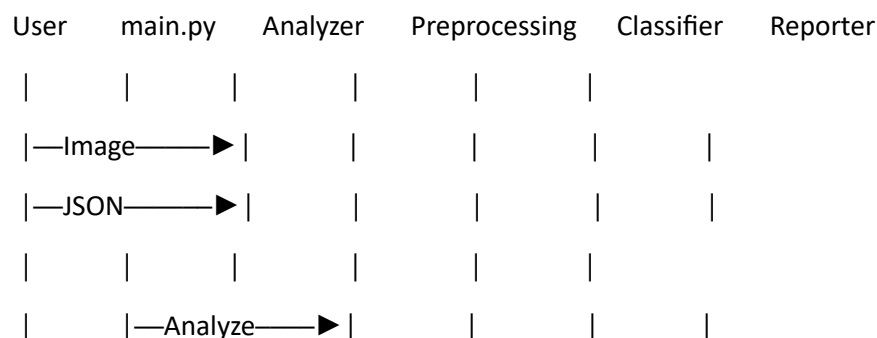
Finally, the system generates the annotated parking-lot image and CSV reports.

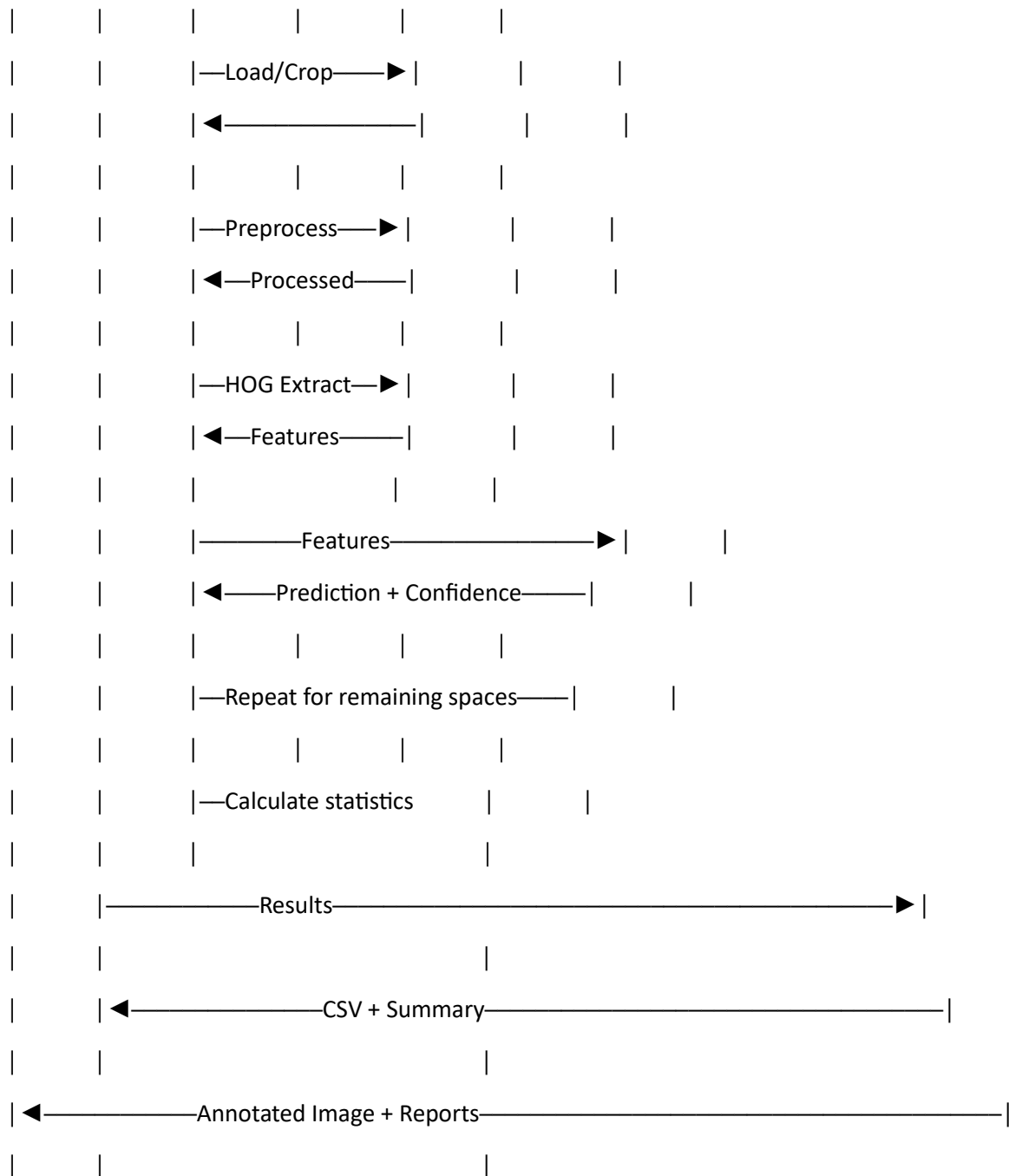
## 7.3 Sequence Diagram

### 7.3.1 Purpose

The Sequence Diagram shows the chronological interaction between the user and the major software components during a parking-lot analysis.

### 7.3.2 Sequence





### 7.3.3 Sequence Description

#### Step 1 — Input

The user provides the image and annotation file through the command-line interface.

#### Step 2 — Analysis Request

main.py initializes the classifier, analyzer, and reporter and starts the analysis.

#### Step 3 — Space Extraction

The analyzer reads the annotations and extracts each parking-space crop from the image.

#### Step 4 — Preprocessing

The crop is passed through the preprocessing pipeline.

#### Step 5 — Feature Extraction

HOG features are generated from the processed image.

#### Step 6 — Classification

The feature vector is passed to the trained SVM classifier.

The classifier returns:

- Predicted class.
- Prediction probability/confidence.

#### Step 7 — Aggregation

The analyzer repeats the process for every parking space and calculates the overall parking statistics.

#### Step 8 — Reporting

The reporter generates the detailed CSV and summary CSV.

#### Step 9 — Visualization

The analyzer generates the annotated parking-lot image.

#### Step 10 — Final Output

The user receives the image and report files.

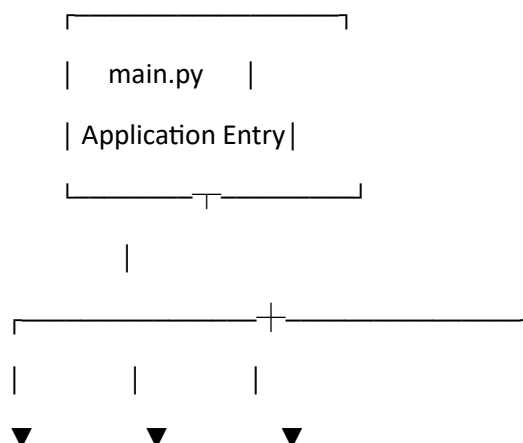
### 7.4 Class / Component Diagram

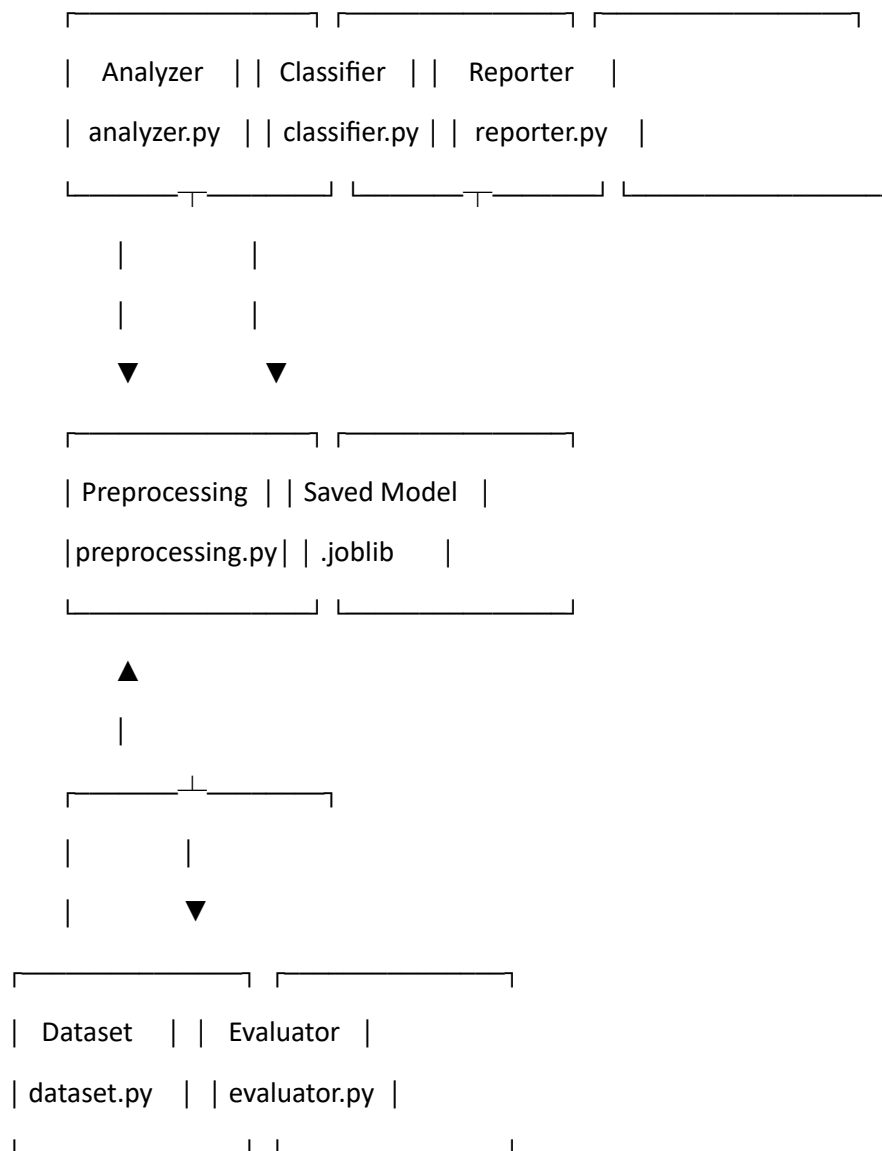
#### 7.4.1 Purpose

The Class/Component Diagram describes the main software modules and the relationships between them.

The project follows a modular architecture with separate components for data processing, preprocessing, classification, evaluation, analysis, and reporting.

#### 7.4.2 Component Diagram





### 7.4.3 Major Classes and Responsibilities

#### ParkingClassifier

**File:** src/classifier.py

Responsible for:

- Creating the SVM classifier.
- Training the model.
- Performing predictions.
- Calculating prediction probabilities.
- Predicting occupancy for an individual image.
- Saving the trained model.

- Loading an existing model.

### **ParkingLotAnalyzer**

**File:** src/analyzer.py

Responsible for:

- Loading annotations.
- Finding parking-space annotations.
- Cropping parking spaces.
- Running predictions.
- Calculating occupancy statistics.
- Drawing annotations on the parking-lot image.

### **ParkingReportGenerator**

**File:** src/reporter.py

Responsible for:

- Saving detailed parking-analysis results.
- Creating summary data.
- Saving parking summary reports.

### **Dataset Processing Functions**

**File:** src/dataset.py

Responsible for:

- Reading COCO annotations.
- Selecting relevant parking-space samples.
- Extracting image crops.
- Creating the processed dataset.
- Creating metadata.

### **Preprocessing Functions**

**File:** src/preprocessing.py

Responsible for:



- Loading images.
- Resizing images.
- Converting images to grayscale.
- Normalizing images.
- Extracting HOG features.

## Evaluation Functions

**File:** src/evaluator.py

Responsible for:

- Loading validation/test data.
- Generating model predictions.
- Calculating evaluation metrics.
- Creating confusion matrices.

## 7.5 ER Diagram

### 7.5.1 Applicability

An **Entity-Relationship (ER) Diagram is not applicable to the current implementation.**

The project does not use a relational database or persistent database storage. Instead, it uses:

- JPG images.
- COCO-format JSON annotations.
- CSV reports.
- A serialized .joblib machine-learning model.

Therefore, there are no database tables, primary keys, foreign keys, or relational entities that need to be represented using an ER diagram.

The project documentation similarly describes file-based inputs and outputs rather than a database-backed architecture.

For the report, the following can be included:

### **ER Diagram: Not Applicable**

The current system does not implement a database layer. Input annotations are stored in COCO-format JSON files, while analysis results are stored as CSV files. The trained machine-learning model is stored as a serialized Joblib file. Consequently, an Entity-Relationship Diagram is not required for the current implementation.

## 7.6 Design Diagram Summary

The diagrams presented in this section describe the system from different perspectives:

| Diagram                 | Purpose                                             |
|-------------------------|-----------------------------------------------------|
| Use Case Diagram        | Shows interaction between the user and system.      |
| Workflow Diagram        | Shows the complete processing pipeline.             |
| Sequence Diagram        | Shows chronological interaction between components. |
| Class/Component Diagram | Shows the modular software architecture.            |
| ER Diagram              | Not applicable because no database is used.         |

Together, these diagrams provide a complete visual representation of the system's **behavior, workflow, component interactions, and software structure**.

## 8. Design Decisions & Rationale

This section explains the major technical and architectural decisions made during the development of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System**. Each decision was made based on the project scope, available dataset, computational requirements, and the need to maintain a modular and understandable implementation.

### 8.1 Selection of Computer Vision Approach

The project uses a **two-stage image-analysis approach**:

1. Identify individual parking-space regions using the provided bounding-box annotations.
2. Classify each extracted parking space as **empty** or **occupied**.

This approach was selected because the dataset provides the locations of individual parking spaces. Instead of attempting to detect parking spaces automatically, the project focuses on the computer-vision classification problem of determining their occupancy status.

This also keeps the project scope clearly defined and allows the HOG + SVM pipeline to be evaluated independently.

### 8.2 Decision to Use HOG Features

#### Decision

**Histogram of Oriented Gradients (HOG)** was selected as the feature-extraction technique.

#### Rationale

HOG represents an image using local gradient and edge-orientation information. This makes it suitable for capturing visual structural differences between an empty parking space and a parking space containing a vehicle.

The project converts each parking-space crop into a standardized **64 × 64 grayscale image** and extracts a **1,764-dimensional HOG feature vector**.

The choice also provides an opportunity to demonstrate a classical computer-vision technique rather than relying entirely on a pre-trained deep-learning model.

### 8.3 Decision to Use SVM

#### Decision

A **Support Vector Machine (SVM)** was selected as the primary classifier.

#### Rationale

The classification problem is binary:

$$\text{Parking Space} \rightarrow \begin{cases} \text{Empty} \\ \text{Occupied} \end{cases}$$

SVM is well suited to classification problems where meaningful features have already been extracted from the input data.

The HOG extractor converts each parking-space image into a fixed-length numerical feature vector, which can then be supplied to the SVM classifier.

The project documentation identifies SVM as the selected classifier for the occupancy-classification stage.

### 8.4 Decision to Use a Calibrated Classifier

#### Decision

The SVM classifier is wrapped using **probability calibration**.

#### Rationale

A standard SVM primarily provides a class prediction. However, the application also benefits from an indication of how confident the model is in its prediction.

Calibration allows the system to obtain probability estimates for the predicted classes.

Therefore, each parking-space prediction can provide:

- Predicted class.
- Estimated probability/confidence.

This makes the output more informative than a simple binary prediction.

### 8.5 Decision to Use COCO Annotations

#### Decision

The project uses the supplied **COCO-format JSON annotations** for parking-space localization.

#### Rationale

The selected dataset is already organized using COCO-style annotations containing image information, categories, and bounding boxes.

Using these annotations provides a standardized way to:

- Identify parking spaces.
- Locate their bounding boxes.
- Extract individual parking-space crops.
- Associate annotations with source images.

It also avoids introducing a separate manual annotation process.

The dataset-processing workflow described in the project documentation is based on these annotated parking-space regions.

## 8.6 Decision to Use a Balanced Dataset Subset

### Decision

A balanced subset of the available parking-space samples was used for model development.

The processed dataset contains:

- **6,000 training samples**
  - 3,000 empty
  - 3,000 occupied
- **2,000 validation samples**
  - 1,000 empty
  - 1,000 occupied
- **2,000 test samples**
  - 1,000 empty
  - 1,000 occupied

Total:

$$6000 + 2000 + 2000 = 10000$$

samples.

### Rationale

Using equal numbers of empty and occupied samples reduces class imbalance within the selected subset.

This makes accuracy easier to interpret because neither class dominates the evaluation dataset.

It also provides a controlled dataset size that is practical for repeated development and testing.

## 8.7 Decision to Separate Training, Validation, and Test Data

### Decision

Three separate dataset splits are maintained:

- Training
- Validation
- Test

### Rationale

Each split serves a different purpose.

#### Training set:

Used to learn the classification model.

#### Validation set:

Used to evaluate the model during development and identify potential performance issues.

#### Test set:

Used for final performance evaluation on data that was not used for model training.

This separation provides a more reliable estimate of the model's generalization performance.

The project documentation explicitly includes separate validation and test evaluation.

## 8.8 Decision to Use 64 × 64 Image Size

### Decision

Each parking-space crop is resized to **64 × 64 pixels** before HOG extraction.

### Rationale

The parking-space images originate from a dataset with consistent but potentially varying crop dimensions.

Using a fixed image size provides:

- Consistent input dimensions.
- Consistent HOG feature dimensions.
- Reduced computational requirements.
- Simpler model training.

The resulting representation produces a fixed **1,764-dimensional feature vector** for every sample.

## 8.9 Decision to Use Modular Project Structure

## Decision

The implementation is divided into multiple Python modules rather than placing the complete system in one file.

The major modules are:

src/

├— dataset.py

├— preprocessing.py

├— classifier.py

├— evaluator.py

├— analyzer.py

└ reporter.py

## Rationale

This separation follows the principle of **separation of responsibilities**.

For example:

- Dataset processing can be changed without modifying the classifier.
- The classifier can be replaced without rewriting the reporting system.
- Evaluation logic is separated from normal application execution.
- Reporting logic is independent of image-processing logic.

This makes the project easier to understand, test, maintain, and extend.

The course project requirements emphasize modular implementation and proper project structure.

## 8.10 Decision to Use File-Based Storage

### Decision

The current implementation uses files instead of a database.

The system stores:

- Input images as JPG files.
- Annotations as JSON.
- Processed metadata as CSV.
- Analysis results as CSV.
- Trained model as a Joblib file.
- Visualization results as image files.

## **Rationale**

A database would introduce additional complexity without providing significant benefits for the current project scope.

The application is primarily a computer-vision analysis tool rather than a multi-user parking-management platform.

Therefore, file-based storage is sufficient for the current implementation.

A database can be introduced later if the project is extended to support persistent parking records, historical occupancy tracking, or multiple parking locations.

## **8.11 Decision to Use CLI Instead of GUI/Web Application**

### **Decision**

The application is implemented as a **command-line application**.

### **Rationale**

The primary objective is to demonstrate the computer-vision and machine-learning pipeline.

Adding a web or graphical interface would introduce additional technologies such as:

- Frontend development.
- Backend APIs.
- Web deployment.
- User-interface design.

These are outside the core scope of the current project.

The CLI keeps the implementation focused on:

**Input → Computer Vision → Classification → Analysis → Output**

A GUI or web dashboard can be considered as a future enhancement.

## **8.12 Decision to Generate Both Visual and Structured Results**

### **Decision**

The system generates both:

1. An annotated parking-lot image.
2. CSV reports.

### **Rationale**

These outputs serve different purposes.

The **annotated image** allows a user to visually inspect which parking spaces were classified as empty or occupied.

The **CSV reports** provide structured information that can be further analyzed using spreadsheet or data-analysis tools.

Using both output types makes the system more useful than producing only a classification score.

### 8.13 Decision to Include Automated Testing

#### Decision

A dedicated tests/ directory is included in the project.

#### Rationale

Computer-vision pipelines contain several stages where errors can occur, such as:

- Incorrect bounding-box coordinates.
- Invalid image loading.
- Incorrect preprocessing.
- Incorrect feature dimensions.
- Missing output files.

Automated tests help verify these individual operations.

The final project test suite contains **7 automated tests**, covering important preprocessing, cropping, sample-data, and output functionality.

The project requirements also call for testing and validation as part of a proper implementation.

### 8.14 Decision to Evaluate Using Multiple Metrics

#### Decision

The classifier is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

#### Rationale

Accuracy alone does not provide enough information about classification behavior.



For example, precision indicates how reliable positive predictions are, while recall indicates how many actual positive cases are correctly identified.

F1-score combines precision and recall into a single measure.

The confusion matrix provides a direct view of:

- Correct empty predictions.
- Incorrect empty predictions.
- Correct occupied predictions.
- Incorrect occupied predictions.

The project's evaluation methodology includes these metrics and confusion matrices.

### 8.15 Decision Summary

| Design Decision            | Selected Approach                   | Main Reason                                                  |
|----------------------------|-------------------------------------|--------------------------------------------------------------|
| Parking-space localization | COCO bounding boxes                 | Already provided by dataset                                  |
| Image representation       | 64 × 64 grayscale                   | Consistent processing                                        |
| Feature extraction         | HOG                                 | Classical CV feature representation                          |
| Classifier                 | SVM                                 | Suitable for fixed feature vectors and binary classification |
| Confidence estimation      | Calibrated classifier               | Provides probability estimates                               |
| Dataset                    | Balanced 10,000-sample subset       | Controlled and balanced development set                      |
| Dataset split              | Train / Validation / Test           | Reliable model evaluation                                    |
| Storage                    | JSON / CSV / JPG / Joblib           | Sufficient for current scope                                 |
| Interface                  | CLI                                 | Keeps focus on CV/ML functionality                           |
| Architecture               | Modular Python files                | Maintainability and testability                              |
| Evaluation                 | Multiple metrics + confusion matrix | More complete performance analysis                           |
| Testing                    | Pytest                              | Automated validation                                         |

### 8.16 Overall Design Rationale

The overall design prioritizes **clarity, modularity, reproducibility, and practical computer-vision implementation.**

Instead of attempting to build a very large system containing unnecessary components, the project focuses on a clearly defined pipeline:

**Annotated Parking Image → Space Extraction → Preprocessing → HOG → SVM → Occupancy Analysis → Visualization and Reports**

This approach keeps the implementation manageable while still demonstrating multiple computer-vision, machine-learning, software-engineering, testing, and documentation concepts required for the evaluated project.

## 9. Implementation Details

### 9.1 Implementation Overview

The **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** is implemented as a modular Python application. The implementation separates dataset preparation, image processing, feature extraction, machine-learning classification, evaluation, parking-lot analysis, and report generation into independent modules.

The overall implementation follows:

Dataset



Annotation Processing



Parking-Space Cropping



Image Preprocessing



HOG Feature Extraction



SVM Training



Model Evaluation



Parking-Lot Analysis



Visualization + Reports

The project structure contains multiple source modules, scripts, tests, configuration files, and documentation, supporting the modular implementation expected for the evaluated project.

## 9.2 Development Environment

The system is implemented using **Python** and uses open-source libraries for computer vision, machine learning, data processing, visualization, model serialization, and testing.

### Main Technologies

| Technology   | Purpose                                                |
|--------------|--------------------------------------------------------|
| Python       | Main programming language                              |
| OpenCV       | Image loading, processing, cropping, and visualization |
| NumPy        | Numerical and array operations                         |
| Pandas       | Dataset metadata and CSV reports                       |
| Scikit-image | HOG feature extraction                                 |
| Scikit-learn | SVM classification and evaluation                      |
| Matplotlib   | Evaluation visualization                               |
| Joblib       | Saving and loading the trained model                   |
| Pytest       | Automated testing                                      |
| JSON         | COCO annotation processing                             |
| Git          | Version control                                        |

The project dependencies are maintained in requirements.txt.

## 9.3 Project Directory Structure

The final implementation follows the following structure:

parking-occupancy-cv/

|

├— README.md

├— statement.md

├— requirements.txt

├— .gitignore

├— config.py

├— main.py

|

```
└─ src/
|   └─ __init__.py
|   └─ dataset.py
|   └─ preprocessing.py
|   └─ classifier.py
|   └─ evaluator.py
|   └─ analyzer.py
|   └─ reporter.py
|
└─ scripts/
|   └─ create_sample.py
|   └─ inspect_dataset.py
|   └─ train_model.py
|
└─ data/
|   └─ sample/
|       └─ sample_parking_lot.jpg
|       └─ annotations.json
|
└─ models/
|   └─ parking_classifier.joblib
|
└─ outputs/
|   └─ parking_analysis.jpg
|   └─ parking_analysis.csv
|   └─ parking_summary.csv
|   └─ validation_confusion_matrix.png
|   └─ test_confusion_matrix.png
|
└─ tests/
```

```
└─ conftest.py
└─ test_project.py
```

The raw and processed datasets and the generated model are kept outside the source-code portion of the repository through the project's .gitignore configuration.

## 9.4 Configuration Module

**File:** config.py

The configuration module centralizes important project parameters and directory locations.

Important configuration values include:

- Raw dataset directory.
- Processed dataset directory.
- Output directory.
- Model directory.
- Image size.
- Maximum number of samples.
- Test split size.
- Random state.
- Model file location.

For example:

```
IMAGE_SIZE = (64, 64)
```

```
MAX_SAMPLES = 3000
```

```
TEST_SIZE = 0.20
```

```
RANDOM_STATE = 42
```

Centralizing these values prevents different modules from using inconsistent settings.

## 9.5 Dataset Preparation

**File:** src/dataset.py

The dataset module is responsible for converting the original annotated dataset into a format suitable for model training.

The source dataset is organized into:

```
train/
```

```
valid/
```

test/

Each split contains:

- Images.
- \_annotations.coco.json.

The COCO annotation file provides:

- Image information.
- Category information.
- Bounding-box annotations.

The project uses the relevant parking-space categories:

1 → space-empty

2 → space-occupied

The spaces category is not used for the occupancy-classification dataset.

### 9.5.1 Dataset Cropping

For each valid annotation, the system obtains the COCO bounding box:

[x, y, width, height]

The corresponding region is cropped from the source image.

The resulting structure is:

data/processed/

|

├─ train/

| └─ empty/

| └─ occupied/

|

├─ valid/

| └─ empty/

| └─ occupied/

|

└─ test/

└─ empty/

└─ occupied/

This organization allows the classifier to determine the class label directly from the directory in which each crop is stored.

## 9.6 Processed Dataset

A balanced subset was created for model development.

| Dataset Split | Empty        | Occupied     | Total         |
|---------------|--------------|--------------|---------------|
| Training      | 3,000        | 3,000        | 6,000         |
| Validation    | 1,000        | 1,000        | 2,000         |
| Test          | 1,000        | 1,000        | 2,000         |
| <b>Total</b>  | <b>5,000</b> | <b>5,000</b> | <b>10,000</b> |

Therefore, the complete processed subset contains:

10,000 parking-space samples

with equal representation of the two classes.

The project documentation describes the dataset preparation and train/validation/test organization used by the system.

## 9.7 Image Preprocessing

**File:** src/preprocessing.py

Each parking-space crop passes through the same preprocessing pipeline before feature extraction.

Original Parking-Space Crop

|



Resize 64 × 64

|



Convert to Grayscale

|



Normalize Pixels

|



Preprocessed Image

### Resize

All crops are resized to:

$$64 \times 64$$

This ensures that every sample has the same dimensions.

### Grayscale Conversion

The color image is converted into a single-channel grayscale image.

This reduces the dimensionality of the input and focuses the feature-extraction stage on image structure and gradients.

### Normalization

Pixel values are normalized to a consistent numerical range before HOG extraction.

## 9.8 HOG Feature Extraction

After preprocessing, HOG features are extracted from each image.

The implementation uses:

Orientations = 9

Pixels per cell = (8, 8)

Cells per block = (2, 2)

Block normalization = L2-Hys

The resulting feature vector has:

$$1764$$

dimensions.

Therefore, the machine-learning classifier does not directly receive the raw  $64 \times 64$  image. Instead, it receives the corresponding HOG feature representation.

$64 \times 64$  Image

↓

HOG

↓

1764 Features

This feature-extraction approach is part of the documented machine-learning methodology of the project.



## 9.9 Model Training

**File:** src/classifier.py

The classifier module implements the parking occupancy model.

The selected classifier is an **SVM with an RBF kernel**.

The main model configuration uses:

Kernel = RBF

C = 10.0

Gamma = scale

The SVM receives the HOG feature vectors and their corresponding labels.

Training can be represented as:

Training Images



Preprocessing



HOG Features



Training Labels



RBF SVM



Calibrated Classifier



Saved Model

The trained model is saved using **Joblib** so that it can later be loaded for inference.

## 9.10 Model Prediction

Once the model has been trained, the saved classifier can be loaded by the main application.

For a new parking-space crop:

Parking-Space Crop



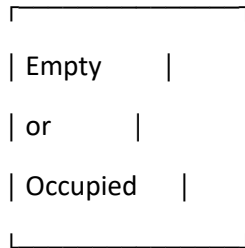
Preprocessing



HOG Feature Extraction



SVM Prediction



The classifier also provides a confidence/probability estimate through the calibrated model.

### 9.11 Parking-Lot Analysis

**File:** `src/analyzer.py`

The `ParkingLotAnalyzer` class is responsible for applying the trained model to a complete parking-lot image.

The process is:

1. Load the parking-lot image.
2. Load the annotation JSON.
3. Find all relevant parking-space annotations.
4. Crop each parking-space region.
5. Preprocess each crop.
6. Extract HOG features.
7. Pass the features to the classifier.
8. Store the prediction and confidence.
9. Draw the prediction on the original image.
10. Calculate overall statistics.

The analyzer repeats this process for every parking space in the input image.

### 9.12 Occupancy Calculation

After all parking spaces have been classified, the analyzer calculates the overall occupancy.

The following values are obtained:

- Total spaces.
- Occupied spaces.
- Empty spaces.
- Occupancy percentage.

The calculation is:

$$\text{Occupancy Percentage} = \frac{\text{Occupied Spaces}}{\text{Total Spaces}} \times 100$$

For the sample application run:

Total Spaces : 100

Occupied Spaces : 59

Empty Spaces : 41

Occupancy : 59.00%

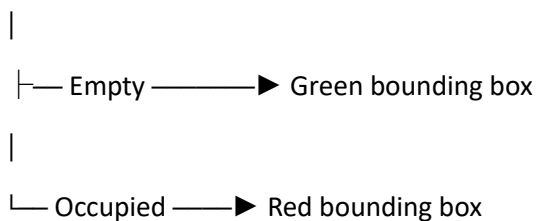
The resulting values are also written to the summary CSV file.

### 9.13 Visualization

The analyzer generates an annotated copy of the original parking-lot image.

The predicted status determines the bounding-box annotation:

Prediction



This allows the user to visually inspect the model's predictions.

The original source image remains unchanged; the annotated image is saved separately in the `outputs/` directory.

### 9.14 Report Generation

**File:** `src/reporter.py`

The `ParkingReportGenerator` class is responsible for converting the analysis results into structured files.

#### Detailed Report

The detailed CSV stores the results associated with individual parking spaces.

### Summary Report

The summary CSV contains:

total\_spaces

occupied\_spaces

empty\_spaces

occupancy\_percentage

Separating report generation from analysis makes the reporting functionality independent and reusable.

## 9.15 Model Evaluation

**File:** src/evaluator.py

The classifier was evaluated on both validation and test datasets.

### Validation Results

| Metric | Score |
|--------|-------|
|--------|-------|

|          |        |
|----------|--------|
| Accuracy | 96.05% |
|----------|--------|

|           |        |
|-----------|--------|
| Precision | 93.90% |
|-----------|--------|

|        |        |
|--------|--------|
| Recall | 98.50% |
|--------|--------|

|          |        |
|----------|--------|
| F1-score | 96.14% |
|----------|--------|

### Test Results

| Metric | Score |
|--------|-------|
|--------|-------|

|          |               |
|----------|---------------|
| Accuracy | <b>96.50%</b> |
|----------|---------------|

|           |               |
|-----------|---------------|
| Precision | <b>95.32%</b> |
|-----------|---------------|

|        |               |
|--------|---------------|
| Recall | <b>97.80%</b> |
|--------|---------------|

|          |               |
|----------|---------------|
| F1-score | <b>96.54%</b> |
|----------|---------------|

The final test classification report also shows strong performance for both empty and occupied classes. The project uses confusion matrices alongside these metrics for a more detailed evaluation.

## 9.16 Testing Implementation

The project uses **Pytest** for automated testing.

The test suite contains **7 tests** covering important project functionality.

The tests verify:

1. Bounding-box cropping.
2. Image preprocessing.
3. HOG feature extraction.
4. Sample annotation availability.
5. Sample image availability.
6. Generated output image.
7. Generated CSV report.

The final test execution produced:

7 passed

This provides automated verification of important components rather than relying solely on manual execution.

### 9.17 Command-Line Interface

The application is executed using main.py.

The general command is:

```
python main.py --image "data/sample/sample_parking_lot.jpg" --annotations  
"data/sample/annotations.json"
```

Optional arguments allow the user to specify output locations.

For example:

--image

--annotations

--output

--csv

This keeps the application simple to operate while allowing different input images and annotation files to be supplied.

### 9.18 Error Handling

The implementation contains validation checks at important stages.

Examples include:

- Checking whether an image can be loaded.

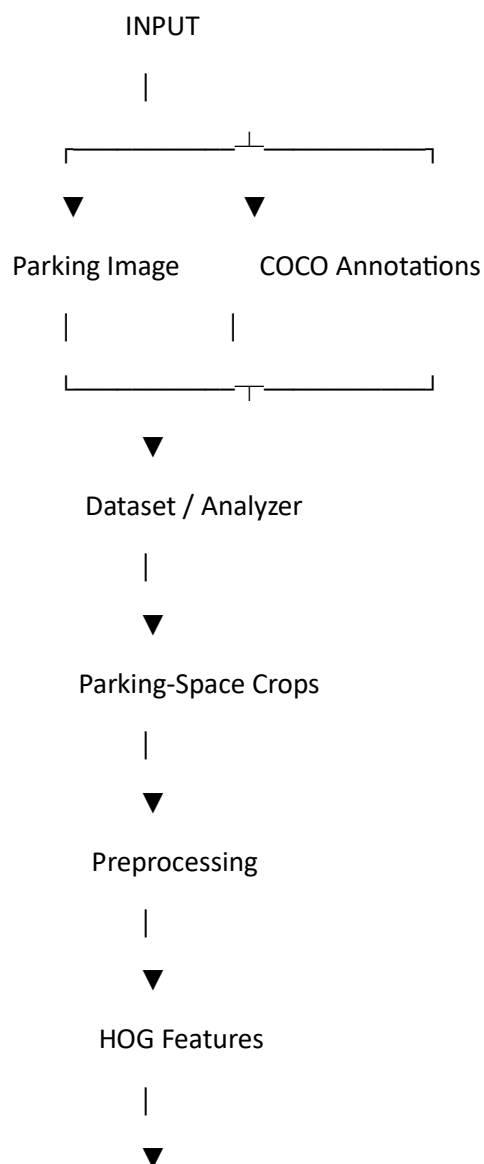
- Checking whether required annotations exist.
- Validating bounding-box crops.
- Checking that training features and labels have matching lengths.
- Handling invalid image files.
- Checking for empty training data.

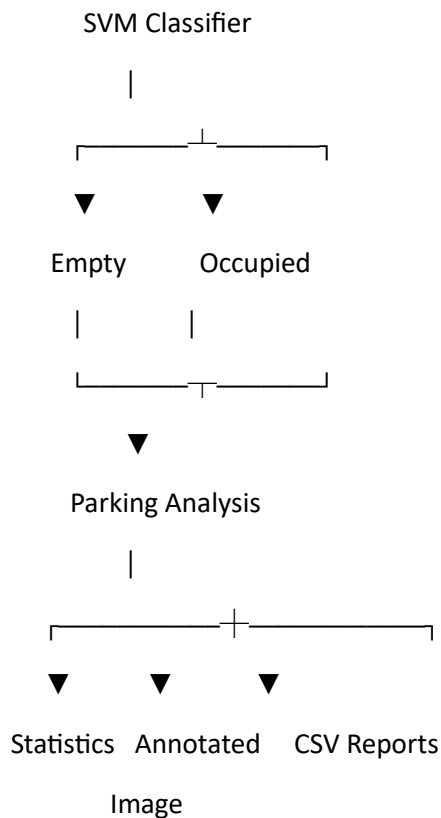
For example, the classifier validates that the number of feature vectors matches the number of labels before training.

This prevents common data-processing errors from silently producing invalid models.

### 9.19 Complete Implementation Pipeline

The final implementation can be summarized as follows:





## 9.20 Implementation Summary

The implementation converts the project's design into a complete working Python application.

The major implementation stages are:

- Dataset and annotation processing.
- Parking-space extraction.
- Image preprocessing.
- HOG feature extraction.
- SVM model training.
- Model evaluation.
- Parking-space prediction.
- Occupancy calculation.
- Image annotation.
- CSV report generation.
- Automated testing.

The implementation therefore covers the complete workflow from **raw annotated parking images to evaluated machine-learning predictions and final user-readable results**.

## 10. Screenshots / Results

### 10.1 Overview

This section presents the results obtained from the implementation of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System**.

The results demonstrate the complete system workflow, including:

- Dataset preparation.
- HOG feature extraction.
- SVM model training.
- Model evaluation.
- Parking-lot occupancy analysis.
- Annotated image generation.
- Automated testing.

The project evaluation methodology includes quantitative classification metrics and confusion matrices, while the application also produces visual and structured outputs.

### 10.2 Dataset Preparation

The original annotated dataset was processed into a balanced subset for model development.

The final processed dataset contains **10,000 parking-space samples**:

| Split        | Empty Occupied Total |              |               |
|--------------|----------------------|--------------|---------------|
| Training     | 3,000                | 3,000        | 6,000         |
| Validation   | 1,000                | 1,000        | 2,000         |
| Test         | 1,000                | 1,000        | 2,000         |
| <b>Total</b> | <b>5,000</b>         | <b>5,000</b> | <b>10,000</b> |



```

• (venv) PS C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv> python src/dataset.py

=====
Processing TRAIN dataset
=====
Selected samples: 6000
Processed 500/6000
Processed 1000/6000
Processed 1500/6000
Processed 2000/6000
Processed 2500/6000
Processed 3000/6000
Processed 3500/6000
Processed 4000/6000
Processed 4500/6000
Processed 5000/6000
Processed 5500/6000
Processed 6000/6000
Completed train

=====
Processing VALID dataset
=====
Selected samples: 2000
Processed 500/2000
Processed 1000/2000
Processed 1500/2000
Processed 2000/2000
Completed valid

=====
Processing TEST dataset
=====
Selected samples: 2000
Processed 500/2000
Processed 1000/2000
Processed 1500/2000
Processed 2000/2000
Completed test

=====
DATASET PREPARATION COMPLETE
=====
Total crops: 10000
Metadata: C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv\data\processed\metadata.csv

Class distribution:
split  label
test   empty    1000
       occupied  1000
train  empty    3000
       occupied  3000
valid  empty    1000
       occupied  1000
dtype: int64

```

Figure 10.1: Dataset preparation showing the processed parking-space samples and class distribution.

### 10.3 Model Training

The classifier was trained using **6,000 training samples**.

Each parking-space image was resized to  $64 \times 64$  pixels and converted into a **1,764-dimensional HOG feature vector** before being supplied to the SVM classifier.

```

• (venv) PS C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv\src> python classifier.py
=====
PARKING OCCUPANCY CLASSIFIER
=====

Extracting features from 3000 empty images...
Processed 500/3000 empty images
Processed 1000/3000 empty images
Processed 1500/3000 empty images
Processed 2000/3000 empty images
Processed 2500/3000 empty images
Processed 3000/3000 empty images

Extracting features from 3000 occupied images...
Processed 500/3000 occupied images
Processed 1000/3000 occupied images
Processed 1500/3000 occupied images
Processed 2000/3000 occupied images
Processed 2500/3000 occupied images
Processed 3000/3000 occupied images

Training feature matrix: (6000, 1764)
Training labels: (6000,)
Training SVM classifier...
Training samples: 6000
Feature dimensions: 1764
Training completed.
Model saved to: C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv\models\parking_classifier.joblib

=====
CLASSIFIER CREATION COMPLETE
=====

```

Figure 10.2: HOG feature extraction and SVM model training.

## 10.4 Model Evaluation

The trained classifier was evaluated separately on the validation and test datasets.

### Validation Performance

| Metric    | Result |
|-----------|--------|
| Accuracy  | 96.05% |
| Precision | 93.90% |
| Recall    | 98.50% |
| F1-score  | 96.14% |

### Test Performance

| Metric    | Result |
|-----------|--------|
| Accuracy  | 96.50% |
| Precision | 95.32% |
| Recall    | 97.80% |
| F1-score  | 96.54% |

The test results demonstrate that the classifier can effectively distinguish between empty and occupied parking-space images in the selected test dataset.

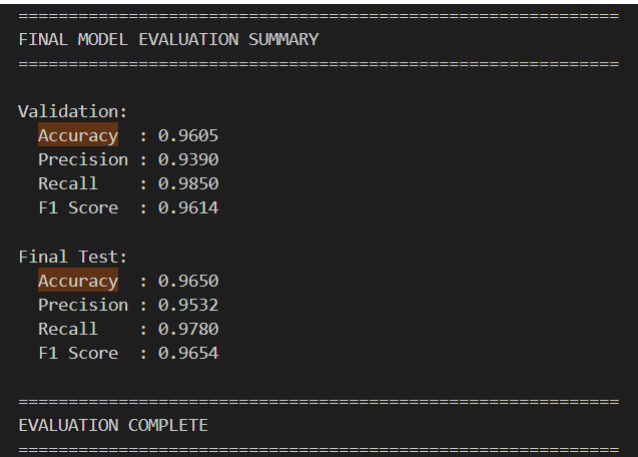


Figure 10.3: Validation and test evaluation results of the parking-space occupancy classifier.

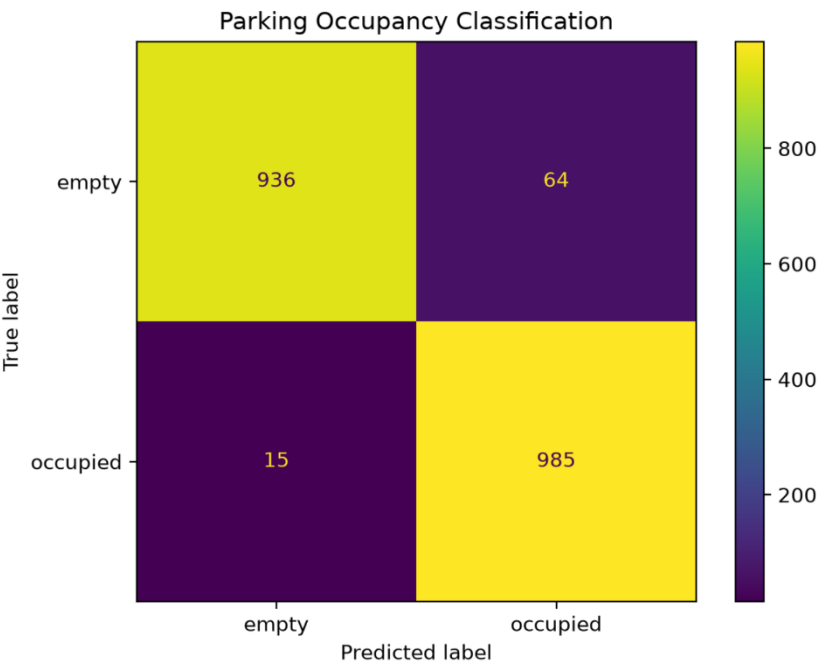
10.5 Confusion Matrix

A confusion matrix was generated for the test dataset to provide a more detailed view of classification performance.

The test dataset contains:

- 1,000 empty samples.
- 1,000 occupied samples.

The confusion matrix allows the model's correct and incorrect predictions for both classes to be examined.



*Figure 10.4: Confusion matrix showing the classification performance of the final model on the test dataset.*

The confusion matrix complements the aggregate accuracy, precision, recall, and F1-score by showing how predictions are distributed between the two classes.

## 10.6 Application Execution

The final application was executed using a sample parking-lot image and its corresponding parking-space annotations.

The application successfully processed **100 parking spaces**.

The resulting output was:

Total Spaces : 100

Occupied Spaces : 59

Empty Spaces : 41

Occupancy : 59.00%

The occupancy percentage is calculated as:

$$\text{Occupancy Percentage} = \frac{59}{100} \times 100 = 59\%$$

```
=====
PARKING LOT OCCUPANCY ANALYSIS SYSTEM
=====

Input image : data\sample\2012-11-06_13_43_31_jpg.rf.00207754dbce0d625d24ca31898fcfd7.jpg
Annotations : data\sample\annotations.json
Model       : C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv\models\parking_classifier.joblib

Loading trained classifier...
Model loaded from: C:\Users\ragha\OneDrive\Desktop\parking-occupancy-cv\models\parking_classifier.joblib

Analyzing parking spaces...

=====
ANALYSIS RESULTS
=====
Total parking spaces : 100
Occupied spaces      : 59
Empty spaces         : 41
Occupancy percentage : 59.00%

=====
OUTPUT FILES
=====
Annotated image : outputs\parking_analysis.jpg
CSV report      : outputs\parking_analysis.csv
Summary report  : outputs\parking_summary.csv

=====
ANALYSIS COMPLETE
=====
```

*Figure 10.5: Successful execution of the parking occupancy analysis application.*

## 10.7 Annotated Parking-Lot Image

The system generates an annotated copy of the input parking-lot image.

Each parking-space prediction is represented using a bounding box:

- **Green:** predicted empty.
- **Red:** predicted occupied.

This provides an immediate visual representation of the model's predictions.

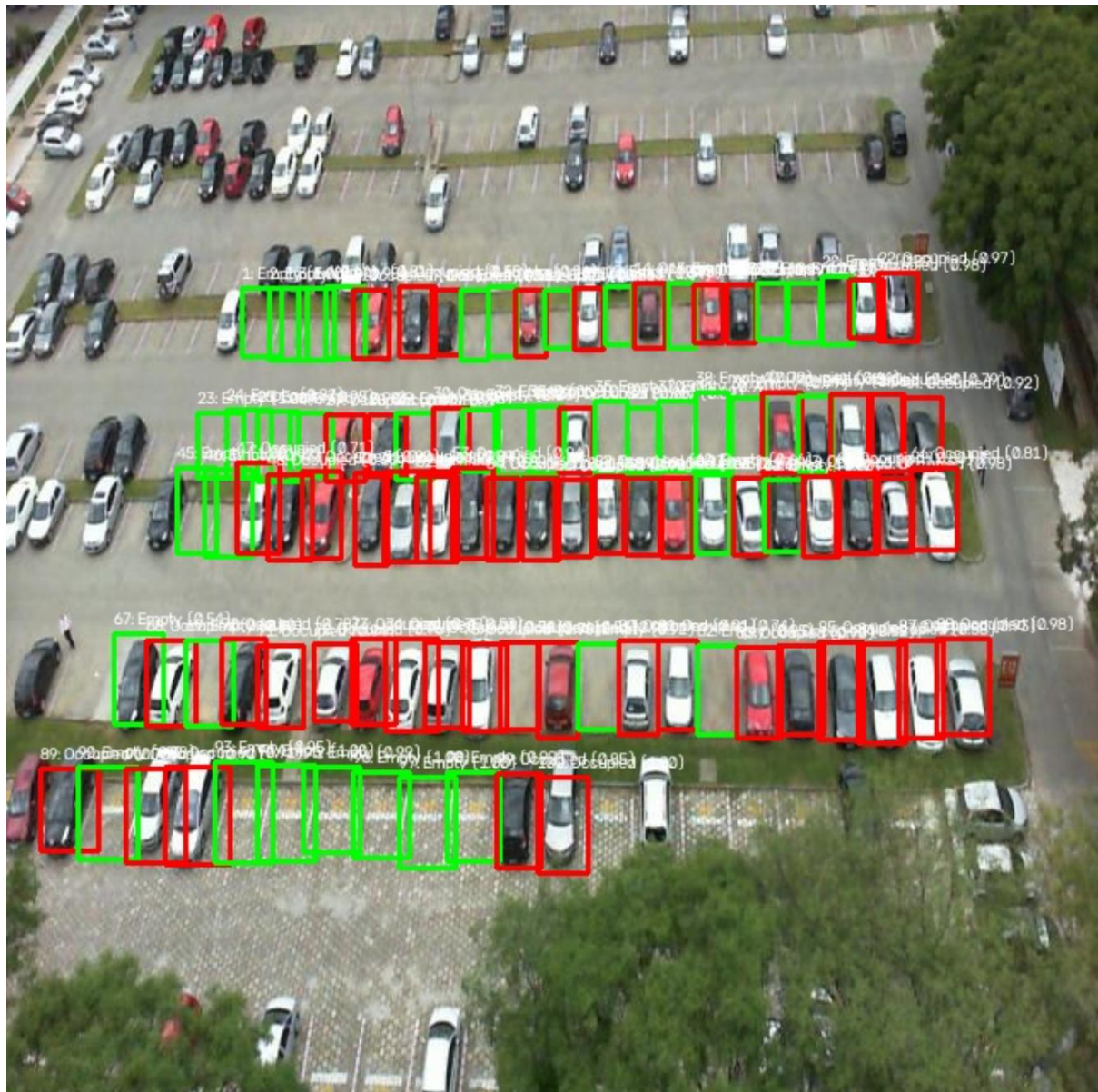


Figure 10.6: Generated parking-lot image with individual parking-space occupancy predictions.

## 10.8 Generated CSV Reports

In addition to the annotated image, the application generates structured CSV outputs.

### Detailed Analysis Report

The detailed report contains information about individual parking-space predictions.

**File:**

outputs/parking\_analysis.csv

**Summary Report**

The summary report contains the aggregate parking statistics:

outputs/parking\_summary.csv

The summary contains:

| Parameter            | Value  |
|----------------------|--------|
| Total Spaces         | 100    |
| Occupied Spaces      | 59     |
| Empty Spaces         | 41     |
| Occupancy Percentage | 59.00% |

This provides both **space-level information** and **parking-lot-level information**.

The project's documented workflow includes generation of the annotated image and CSV-based analysis outputs.

## 10.9 Automated Test Results

The project includes a Pytest-based automated test suite covering important parts of the implementation.

The final test execution produced:

7 passed

The tests cover:

1. Bounding-box cropping.
2. Image preprocessing.
3. HOG feature extraction.
4. Sample annotation availability.
5. Sample image availability.
6. Output image generation.
7. CSV report generation.

```
(venv) PS C:\Users\rasha\OneDrive\Desktop\parking-occupancy-cv> pytest -v
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\rasha\OneDrive\Desktop\parking-occupancy-cv\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rasha\OneDrive\Desktop\parking-occupancy-cv
collected 7 items

tests/test_project.py::test_crop_bounding_box PASSED [ 14%]
tests/test_project.py::test_preprocess_image PASSED [ 28%]
tests/test_project.py::test_hog_feature_extraction PASSED [ 42%]
tests/test_project.py::test_sample_annotations PASSED [ 57%]
tests/test_project.py::test_sample_image_exists PASSED [ 71%]
tests/test_project.py::test_output_image_exists PASSED [ 85%]
tests/test_project.py::test_output_csv_exists PASSED [100%]

===== 7 passed in 13.65s =====
```

Figure 10.7: Automated project tests successfully passing.

## 10.10 Overall Results

The final implementation successfully demonstrates the complete parking-occupancy analysis pipeline.

| Result Category       | Outcome          |
|-----------------------|------------------|
| Processed Samples     | 10,000           |
| Training Samples      | 6,000            |
| Validation Samples    | 2,000            |
| Test Samples          | 2,000            |
| HOG Features          | 1,764 per sample |
| Test Accuracy         | <b>96.50%</b>    |
| Test Precision        | <b>95.32%</b>    |
| Test Recall           | <b>97.80%</b>    |
| Test F1-score         | <b>96.54%</b>    |
| Sample Parking Spaces | 100              |
| Sample Occupied       | 59               |
| Sample Empty          | 41               |
| Sample Occupancy      | <b>59.00%</b>    |
| Automated Tests       | <b>7 passed</b>  |

## 10.11 Result Discussion

The results indicate that the implemented HOG + SVM pipeline performs well for the selected parking-space classification task.

The **96.50% test accuracy** indicates that the majority of the test parking-space samples were classified correctly. The **97.80% recall** for the occupied class is particularly relevant to the application because it indicates that most occupied parking spaces in the test set were successfully identified.

The final application also successfully converts individual predictions into an understandable parking-lot representation through the annotated image and structured CSV reports.

The results therefore demonstrate both:

1. **Machine-learning performance** through quantitative evaluation.
2. **Application functionality** through visualizations, occupancy statistics, reports, and automated tests.

The project documentation identifies the same evaluation metrics and visual output approach as part of the implemented system.

## 11. Testing Approach

### 11.1 Overview

Testing was performed to verify that the major components of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** work correctly and produce the expected outputs.

The testing strategy combines:

- **Unit testing** of individual functions.
- **Integration testing** of interacting modules.
- **Model evaluation** using validation and test datasets.
- **Application-level testing** using a complete parking-lot image.
- **Output validation** for generated images and CSV reports.

The project requirements emphasize validation, error handling, and testing as part of the implementation.

### 11.2 Testing Objectives

The primary objectives of testing were:

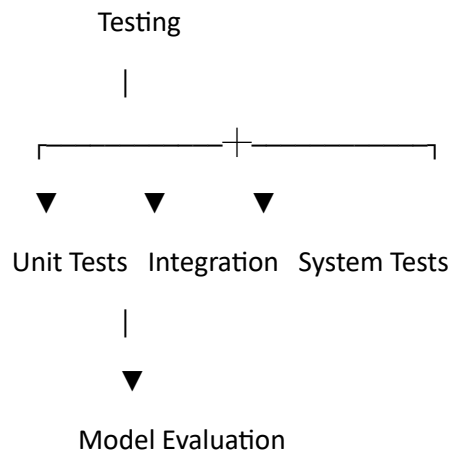
1. Verify that individual modules perform their intended operations.
2. Ensure that image preprocessing produces the expected format.
3. Verify that HOG feature extraction produces consistent feature vectors.
4. Ensure that parking-space bounding boxes are cropped correctly.
5. Verify that the classifier can produce occupancy predictions.
6. Verify that the complete application can process a parking-lot image.



7. Ensure that expected output files are generated.
8. Evaluate the classification performance using unseen test data.
9. Detect implementation errors before final submission.

### 11.3 Testing Levels

The project uses multiple levels of testing.



#### Unit Testing

Individual functions are tested independently.

#### Integration Testing

Multiple modules are tested together to verify that their outputs and inputs are compatible.

#### System Testing

The complete application is executed using an input image and annotation file.

#### Model Testing

The trained classifier is evaluated using validation and test datasets.

### 11.4 Unit Testing

The project uses **Pytest** for automated unit testing.

The tests are located in:

tests/

├── conftest.py

└── test\_project.py

The test suite contains **7 automated tests**.

#### 11.4.1 Test Case 1 — Bounding-Box Cropping

**Purpose:**

Verify that a parking-space region can be correctly extracted using bounding-box coordinates.

**Input:**

An image and a valid bounding box.

**Expected Result:**

The returned crop should contain a valid image region with non-zero dimensions.

**Status:** Passed.

#### 11.4.2 Test Case 2 — Image Preprocessing

**Purpose:**

Verify that the preprocessing pipeline correctly transforms an input image.

**Operations tested:**

- Resizing.
- Grayscale conversion.
- Pixel normalization.

**Expected Result:**

- Image dimensions should be  $64 \times 64$ .
- Image should have the expected processed representation.
- Pixel values should be normalized.

**Status:** Passed.

#### 11.4.3 Test Case 3 — HOG Feature Extraction

**Purpose:**

Verify that HOG features can be extracted from a valid image.

**Expected Result:**

- HOG extraction should complete successfully.
- The resulting feature vector should have the expected dimensions.

The implemented configuration produces:

1764

features per parking-space sample.

**Status:** Passed.

#### **11.4.4 Test Case 4 — Sample Annotation Availability**

**Purpose:**

Verify that the sample COCO annotation file exists and contains usable parking-space annotations.

**Expected Result:**

- Annotation file is available.
- JSON structure can be read.
- Parking-space annotations are present.

**Status:** Passed.

#### **11.4.5 Test Case 5 — Sample Image Availability**

**Purpose:**

Verify that the sample parking-lot image required for application testing exists.

**Expected Result:**

- Image file exists.
- Image can be accessed by the application.

**Status:** Passed.

#### **11.4.6 Test Case 6 — Output Image Generation**

**Purpose:**

Verify that the parking analysis process generates the expected annotated image.

**Expected Result:**

- Output image is created.
- Generated file is accessible after execution.

**Status:** Passed.

#### **11.4.7 Test Case 7 — CSV Report Generation**

**Purpose:**

Verify that the analysis process generates the expected CSV report.

**Expected Result:**

- CSV file is created.
- The file contains the expected analysis information.

**Status:** Passed.

### 11.5 Unit Test Summary

| Test ID | Test Case             | Expected Result             | Status |
|---------|-----------------------|-----------------------------|--------|
| TC-01   | Bounding-box cropping | Valid image crop            | Passed |
| TC-02   | Image preprocessing   | 64 × 64 processed image     | Passed |
| TC-03   | HOG extraction        | 1,764-dimensional vector    | Passed |
| TC-04   | Sample annotations    | Valid annotations available | Passed |
| TC-05   | Sample image          | Image available             | Passed |
| TC-06   | Output image          | Annotated image generated   | Passed |
| TC-07   | CSV report            | CSV report generated        | Passed |

**Total Tests:** 7

**Passed:** 7

**Failed:** 0

### 11.6 Model Evaluation Testing

Testing was also performed at the machine-learning level.

The trained model was evaluated using separate validation and test datasets.

#### Validation Dataset

| Metric | Result |
|--------|--------|
|--------|--------|

|          |        |
|----------|--------|
| Accuracy | 96.05% |
|----------|--------|

|           |        |
|-----------|--------|
| Precision | 93.90% |
|-----------|--------|

|        |        |
|--------|--------|
| Recall | 98.50% |
|--------|--------|

|          |        |
|----------|--------|
| F1-score | 96.14% |
|----------|--------|

#### Test Dataset

| Metric | Result |
|--------|--------|
|--------|--------|

|          |               |
|----------|---------------|
| Accuracy | <b>96.50%</b> |
|----------|---------------|

|           |               |
|-----------|---------------|
| Precision | <b>95.32%</b> |
|-----------|---------------|

|        |               |
|--------|---------------|
| Recall | <b>97.80%</b> |
|--------|---------------|

|          |               |
|----------|---------------|
| F1-score | <b>96.54%</b> |
|----------|---------------|

The use of separate validation and test data provides a way to assess whether the trained model generalizes beyond its training samples. The project evaluation methodology includes these classification metrics and confusion matrices.

### 11.7 Confusion-Matrix Testing

Confusion matrices were generated for both validation and test datasets.

The confusion matrix provides four types of outcomes:

| <b>Actual</b> | <b>Predicted</b> | <b>Meaning</b> |
|---------------|------------------|----------------|
|---------------|------------------|----------------|

|       |       |                          |
|-------|-------|--------------------------|
| Empty | Empty | Correct empty prediction |
|-------|-------|--------------------------|

|       |          |                          |
|-------|----------|--------------------------|
| Empty | Occupied | Incorrect classification |
|-------|----------|--------------------------|

|          |       |                          |
|----------|-------|--------------------------|
| Occupied | Empty | Incorrect classification |
|----------|-------|--------------------------|

|          |          |                             |
|----------|----------|-----------------------------|
| Occupied | Occupied | Correct occupied prediction |
|----------|----------|-----------------------------|

This provides more detailed information than accuracy alone.

The test dataset contained:

- 1,000 empty samples.
- 1,000 occupied samples.

The resulting confusion matrix was saved as:

outputs/test\_confusion\_matrix.png

### 11.8 Integration Testing

Integration testing was performed by connecting multiple modules and verifying that they work together.

The main integrated pipeline is:

Input Image

+

Annotations

↓

ParkingLotAnalyzer

↓

Image Cropping

↓

Preprocessing



HOG Extraction



ParkingClassifier



Occupancy Prediction



Parking Statistics



ParkingReportGenerator



Annotated Image + CSV Reports

This verifies that the output produced by one component is correctly accepted by the next component.

For example:

- The analyzer produces an image crop.
- The preprocessing module processes the crop.
- HOG converts it into a feature vector.
- The classifier accepts the feature vector.
- The analyzer receives the prediction.
- The reporter receives the resulting analysis data.

## 11.9 System Testing

The complete application was tested using a sample parking-lot image containing **100 annotated parking spaces**.

The application successfully produced:

Total Spaces : 100

Occupied Spaces : 59

Empty Spaces : 41

Occupancy : 59.00%

The system generated:

outputs/parking\_analysis.jpg

outputs/parking\_analysis.csv

outputs/parking\_summary.csv

This confirms that the complete pipeline can successfully process an input image and produce the expected visual and structured outputs.

### 11.10 Error Handling Tests

The implementation also includes checks for common invalid conditions.

Examples include:

#### Invalid Image

If an image cannot be loaded, the system should identify the failure rather than continuing with an invalid image.

#### Invalid Bounding Box

Bounding-box processing checks whether a usable crop can be generated.

#### Empty Training Data

The classifier checks that training data is available before attempting model training.

#### Feature/Label Mismatch

Before training, the classifier verifies that the number of feature vectors matches the number of labels.

#### Invalid Input Data

The dataset and analysis components include validation around file loading and image processing.

These checks reduce the possibility of silent failures and invalid model results.

### 11.11 Testing Results

The final testing stage produced the following overall result:

| Testing Area           | Result     |
|------------------------|------------|
| Unit Tests             | 7/7 Passed |
| Image Preprocessing    | Passed     |
| HOG Feature Extraction | Passed     |
| Bounding-Box Cropping  | Passed     |
| Application Execution  | Passed     |

| Testing Area               | Result                 |
|----------------------------|------------------------|
| Annotated Image Generation | Passed                 |
| CSV Report Generation      | Passed                 |
| Model Validation           | 96.05% Accuracy        |
| Model Testing              | <b>96.50% Accuracy</b> |

The project documentation also includes automated tests as part of the final implementation and reports successful execution of the test suite.

### 11.12 Testing Conclusion

The testing process verified the system at multiple levels, from individual functions to the complete parking-lot analysis workflow.

The **7 automated tests passed successfully**, while the machine-learning evaluation demonstrated strong classification performance on the held-out test dataset.

Therefore, testing confirms that the implemented system can:

- Process parking-space images.
- Extract HOG features.
- Classify parking spaces.
- Calculate occupancy statistics.
- Generate annotated images.
- Generate CSV reports.
- Execute the complete workflow successfully.

This multi-level testing approach provides confidence that the final implementation satisfies the defined functional and quality requirements.

## 12. Challenges Faced

### 12.1 Overview

During the development of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System**, several challenges were encountered during dataset preparation, feature extraction, model development, integration, and testing.

These challenges were important because the project involved not only developing a machine-learning model but also integrating it into a complete computer-vision application.

### 12.2 Understanding and Preparing the Dataset

One of the initial challenges was understanding the structure of the downloaded parking dataset.



The dataset was provided in **COCO annotation format**, where image information, categories, and bounding-box annotations are stored in JSON files.

The dataset contained multiple categories, including:

- spaces
- space-empty
- space-occupied

The project needed only the empty and occupied parking-space categories for the classification task.

Therefore, the dataset-processing module had to correctly:

1. Read the COCO JSON file.
2. Identify the relevant category IDs.
3. Match annotations with their corresponding images.
4. Extract valid bounding boxes.
5. Generate individual parking-space crops.

Incorrect handling of the annotations could have resulted in incorrect training samples.

### 12.3 Handling Bounding-Box Coordinates

Another challenge was correctly interpreting the COCO bounding-box format.

COCO represents a bounding box as:

$$[x, y, w, idthheight]$$

where:

- $x$ = horizontal starting coordinate.
- $y$ = vertical starting coordinate.
- $width$ = width of the bounding box.
- $height$ = height of the bounding box.

These values had to be converted correctly into image slicing coordinates.

The cropping process also needed to account for invalid or unusable bounding boxes.

A separate test was therefore implemented to verify the cropping functionality.

### 12.4 Maintaining Consistent Preprocessing

A significant implementation challenge was ensuring that the **same preprocessing pipeline** was used during both training and prediction.

The classifier was trained using:

Image



Resize



Grayscale



Normalize



HOG



SVM

If prediction used a different preprocessing sequence, the feature vectors supplied to the classifier could become inconsistent with the features used during training.

Therefore, preprocessing functionality was centralized in `preprocessing.py` and reused by the training and prediction pipelines.

### 12.5 Avoiding Duplicate Preprocessing

During development, an issue occurred where an image could be preprocessed before being passed to a function that performed preprocessing again.

This resulted in inconsistent processing between different parts of the application.

The problem was resolved by ensuring that the analyzer performs preprocessing exactly once before HOG extraction.

The final pipeline is:

Original Crop



Preprocess Once



HOG



SVM

This helped ensure consistency between training and inference.

## 12.6 Feature-Dimension Consistency

The HOG configuration generates a fixed-length feature vector.

For the project's configuration:

$$\text{Feature Size} = 1764$$

The classifier therefore expects every parking-space sample to produce the same feature dimension.

Any difference in image size or HOG configuration could result in an incompatible feature vector.

To avoid this problem, image resizing and HOG parameters were kept consistent throughout the system.

## 12.7 Model Probability and Confidence

A standard SVM prediction provides the predicted class, but the application also required a confidence/probability value for individual predictions.

Therefore, the classifier was implemented using a **calibrated SVM**.

This allowed the system to obtain probability estimates in addition to the predicted occupancy class.

The calibration step made the application output more informative because a prediction could be represented as:

Prediction : Occupied

Confidence : 0.97

rather than only:

Prediction : Occupied

## 12.8 Debugging Incorrect Occupancy Statistics

One of the more important integration issues occurred while calculating the overall parking occupancy.

The classifier internally represents the classes numerically:

0 → Empty

1 → Occupied

However, the analysis logic initially compared the numerical prediction with a text value such as "occupied".

As a result, the individual predictions were being generated, but the aggregate occupancy calculation was incorrect.

The issue was identified by comparing:

- Individual CSV predictions.

- Terminal summary.
- Expected number of occupied spaces.

The mapping was corrected to explicitly convert the numerical prediction:

0 → empty

1 → occupied

After the correction, the application produced the correct sample result:

Total Spaces : 100

Occupied Spaces : 59

Empty Spaces : 41

Occupancy : 59.00%

This demonstrated the importance of testing not only individual modules but also their integration.

## 12.9 Model Evaluation

Another challenge was determining whether the trained classifier was actually performing well rather than relying only on successful program execution.

A program can execute without errors while still producing poor predictions.

Therefore, separate validation and test evaluations were implemented using:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Confusion matrices.

The final test accuracy was:

96.50%

with an F1-score of:

96.54%

This provided quantitative evidence of the model's performance.

## 12.10 Managing Dataset Size

The original dataset contained a very large number of annotated parking-space samples.

Using the entire dataset for every development iteration would increase processing time and make debugging less convenient.

Therefore, a controlled subset was created:

Training : 6,000 samples

Validation : 2,000 samples

Test : 2,000 samples

Total : 10,000 samples

The classes were balanced within each split.

This provided a practical dataset size for development while maintaining separate validation and test data.

### 12.11 Large Model File Size

Another practical challenge was the size of the trained model file.

The generated Joblib model is approximately **128.5 MB**.

This is large enough that directly committing the model to a normal GitHub repository is undesirable for this project.

Therefore, the model file was excluded through .gitignore:

```
models/*.joblib
```

The repository contains the source code and instructions required to train the model rather than storing the large generated model file directly in the Git repository.

### 12.12 Command-Line Integration

The individual modules worked independently, but they also needed to be connected through a single application entry point.

The main.py module was therefore implemented to coordinate:

Classifier

+

Analyzer

+

Reporter

↓

Complete Application

Command-line arguments were added so that users can specify the input image, annotations, and output locations.

This made the project easier to demonstrate and reproduce.

### 12.13 Testing and Debugging

Testing helped identify several implementation-level issues during development.

For example, testing was used to verify:

- Image loading.
- Bounding-box cropping.
- Preprocessing.
- HOG feature dimensions.
- Sample data availability.
- Output image generation.
- CSV generation.

The final test suite successfully passed all **7 automated tests**.

This demonstrated that the project was not evaluated solely through manual execution.

### 12.14 Documentation and Project Organization

Another challenge was organizing the project so that the implementation, documentation, dataset instructions, testing, and outputs were clearly separated.

The final structure separates:

Source Code → src/

Utility Scripts → scripts/

Tests → tests/

Sample Data → data/sample/

Generated Outputs → outputs/

Model Storage → models/

This makes the project easier for another developer or evaluator to understand.

The course requirements emphasize proper folder/package structure, modular implementation, documentation, testing, and version control.

### 12.15 Challenge and Solution Summary

| Challenge | Solution |
|-----------|----------|
|-----------|----------|

|                                |                                                   |
|--------------------------------|---------------------------------------------------|
| Understanding COCO annotations | Implemented dedicated annotation-processing logic |
|--------------------------------|---------------------------------------------------|

| Challenge                 | Solution                                                    |
|---------------------------|-------------------------------------------------------------|
| Extracting parking spaces | Converted COCO bounding boxes into image crops              |
| Inconsistent image inputs | Standardized images to 64 × 64                              |
| Feature inconsistency     | Used a fixed HOG configuration                              |
| Duplicate preprocessing   | Centralized preprocessing and ensured it runs once          |
| Confidence estimation     | Used calibrated SVM probabilities                           |
| Incorrect occupancy count | Corrected numerical class-to-label mapping                  |
| Evaluating model quality  | Added validation/test metrics and confusion matrices        |
| Large dataset             | Created balanced 10,000-sample development subset           |
| Large model file          | Excluded .joblib model from Git                             |
| Module integration        | Connected components through main.py                        |
| Regression/errors         | Added 7 automated Pytest cases                              |
| Project organization      | Separated source, scripts, tests, data, outputs, and models |

## 12.16 Conclusion

The challenges encountered during development were not limited to machine-learning model training. They also involved **dataset interpretation, image processing consistency, software integration, debugging, evaluation, testing, and project organization**.

Resolving these issues resulted in a more reliable and structured implementation. In particular, the debugging of the occupancy-calculation logic demonstrated the importance of validating the complete end-to-end workflow rather than assuming that correct individual predictions automatically guarantee correct final results.

The final implementation successfully integrates the complete pipeline from annotated input images to **parking-space classification, occupancy statistics, annotated visualization, CSV reporting, and automated testing**.

## 13. Learnings & Key Takeaways

### 13.1 Overview

The development of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** provided practical experience in combining computer-vision techniques, machine-learning algorithms, dataset processing, software architecture, testing, and documentation.

The project was not limited to training a classifier. It required the development of a complete pipeline in which raw image data is transformed into meaningful parking-occupancy information.

### 13.2 Understanding the Complete Computer Vision Pipeline

One of the most important learnings was understanding how an end-to-end computer-vision system is constructed.

The project follows:

Input Image



Object/Region Localization



Image Cropping



Preprocessing



Feature Extraction



Machine-Learning Classification



Prediction



Analysis



Visualization + Reports

This demonstrated that a computer-vision application involves considerably more than simply applying a machine-learning model to an image.

### 13.3 Learning HOG Feature Extraction

The project provided practical understanding of **Histogram of Oriented Gradients (HOG)**.

HOG converts image information into a numerical feature representation based primarily on local gradients and edge orientations.

The project uses:

- $64 \times 64$  image input.
- 9 orientations.
- $8 \times 8$  pixels per cell.



- $2 \times 2$  cells per block.
- L2-Hys normalization.

The resulting representation contains **1,764 features** per parking-space sample.

This demonstrated how raw visual information can be converted into a structured feature vector suitable for traditional machine-learning algorithms.

### 13.4 Understanding SVM Classification

The project provided practical experience with **Support Vector Machines** for binary classification.

The classifier determines whether a parking-space image belongs to one of two classes:

0 → Empty

1 → Occupied

The project demonstrated the complete process of:

1. Preparing labeled data.
2. Extracting numerical features.
3. Training an SVM.
4. Saving the trained model.
5. Loading the model.
6. Performing predictions.
7. Obtaining confidence estimates.

This provided practical understanding of how a classical machine-learning classifier can be integrated into a computer-vision application.

### 13.5 Understanding Dataset Preparation

Another important learning was that **dataset preparation is a major part of machine-learning development.**

Before training the classifier, the project required:

- Understanding the annotation format.
- Identifying relevant categories.
- Extracting bounding boxes.
- Cropping individual parking spaces.
- Separating empty and occupied samples.
- Creating training, validation, and test splits.

- Maintaining class balance.

The final processed dataset contained 10,000 samples distributed across the three dataset splits.

This demonstrated that the quality and organization of the input data directly affect the usefulness of the final model.

### 13.6 Importance of Consistent Preprocessing

A key technical learning was that training and prediction must use the **same preprocessing pipeline**.

The project standardizes every parking-space image before HOG extraction.

Resize



Grayscale



Normalize



HOG

If training and inference use different preprocessing operations, the resulting feature representations may not be compatible.

Therefore, preprocessing was implemented as a reusable module rather than duplicating preprocessing logic in different parts of the project.

### 13.7 Understanding Model Evaluation

The project demonstrated why model evaluation should not depend only on accuracy.

The classifier was evaluated using:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Confusion matrix.

The final test results were:

| Metric | Result |
|--------|--------|
|--------|--------|

|          |               |
|----------|---------------|
| Accuracy | <b>96.50%</b> |
|----------|---------------|

| Metric | Result |
|--------|--------|
|--------|--------|

|           |        |
|-----------|--------|
| Precision | 95.32% |
|-----------|--------|

|        |        |
|--------|--------|
| Recall | 97.80% |
|--------|--------|

|          |        |
|----------|--------|
| F1-score | 96.54% |
|----------|--------|

These metrics provide a more complete understanding of model performance than a single accuracy value.

### 13.8 Importance of Validation and Test Data

The project demonstrated the importance of keeping training and evaluation data separate.

The dataset was divided into:

Training → Model learning

Validation → Development evaluation

Test → Final evaluation

This prevents the final performance measurement from being based solely on samples that were used during model training.

It also provides a better indication of how the model performs on unseen data.

### 13.9 Learning from Debugging

One of the most valuable software-engineering lessons came from debugging the occupancy calculation.

The classifier internally used numerical labels:

0 → Empty

1 → Occupied

The analysis logic initially handled the prediction incorrectly when calculating aggregate statistics.

This caused a mismatch between the individual predictions and the final occupancy count.

The issue demonstrated an important principle:

**Correct individual components do not necessarily guarantee a correct end-to-end system.**

Integration testing and checking intermediate outputs were therefore essential.

### 13.10 Importance of Modular Design

The project demonstrated the advantages of separating responsibilities into different modules.

For example:

dataset.py



preprocessing.py



classifier.py



analyzer.py



reporter.py

Each module has a specific responsibility.

This makes the project:

- Easier to understand.
- Easier to test.
- Easier to debug.
- Easier to modify.
- Easier to extend.

The course project requirements also emphasize modular and clean implementation.

### 13.11 Learning the Importance of Automated Testing

The project demonstrated the value of automated testing in a computer-vision application.

Seven automated tests were implemented to verify important functionality.

7 Tests

|

├— Cropping

├— Preprocessing

├— HOG Features

├— Sample Annotations

├— Sample Image

├— Output Image

└— CSV Output

All seven tests passed successfully.

This showed that automated tests can detect implementation problems early and provide confidence when modifying the project.

### 13.12 Learning Git and Project Organization

The project also provided practical experience with software-development practices such as:

- Git repository initialization.
- .gitignore configuration.
- Source-code organization.
- Separating generated files from source files.
- Maintaining project documentation.
- Preparing a repository suitable for GitHub.

The project requirements specifically include Git/GitHub and appropriate repository documentation.

### 13.13 Understanding the Difference Between a Model and an Application

An important conceptual learning was the distinction between a **machine-learning model** and a **complete application**.

The SVM model alone only performs classification.

The complete application additionally needs:

Dataset Processing

+

Preprocessing

+

Feature Extraction

+

Model

+

Analysis

+

Visualization

+

Reporting

+

Testing

Therefore, developing a useful ML application requires software engineering around the model itself.

### 13.14 Understanding the Limitations of the Current Approach

The project also highlighted the limitations of the selected approach.

The current system depends on parking-space annotations to know where individual parking spaces are located.

Therefore, it does not automatically discover parking spaces in a completely new parking-lot image.

Similarly, the current system operates on still images rather than live video.

Recognizing these limitations is important because model performance on a controlled dataset does not automatically imply equivalent performance in every real-world environment.

### 13.15 Key Technical Takeaways

The major technical lessons from the project can be summarized as follows:

1. **Dataset quality and preparation are fundamental to ML projects.**
2. **Consistent preprocessing is essential between training and inference.**
3. **HOG can convert image information into useful fixed-size feature vectors.**
4. **SVM can effectively perform binary classification using engineered features.**
5. **Multiple evaluation metrics provide better insight than accuracy alone.**
6. **Confusion matrices help identify class-specific classification errors.**
7. **Modular architecture improves maintainability and debugging.**
8. **Automated testing is important even for ML/CV applications.**
9. **End-to-end integration must be tested separately from individual modules.**
10. **A trained model is only one component of a complete ML application.**

### 13.16 Overall Learning Outcome

The project provided practical experience in taking a computer-vision problem from an initial problem definition to a working implementation.

The complete learning progression can be represented as:

Problem Definition

↓

Dataset Understanding



Data Preparation



Computer Vision Processing



Feature Engineering



Machine Learning



Model Evaluation



Software Integration



Testing



Documentation

The most important takeaway is that successful computer-vision development requires the combination of **machine learning, image processing, software engineering, testing, and systematic evaluation**, rather than focusing exclusively on model training.

## 14. Future Enhancements

### 14.1 Overview

The current **Computer Vision Based Parking Lot Occupancy Detection and Analysis System** successfully performs parking-space occupancy classification using annotated parking-lot images, HOG feature extraction, and an SVM classifier.

However, the current implementation has several limitations that provide opportunities for further development. The future enhancements described below focus on improving **automation, accuracy, scalability, real-time capability, and practical usability**.

The project's documented future-improvement areas include automatic parking-space detection, real-time/video processing, deep-learning approaches, and a user-facing interface.

## 14.2 Automatic Parking-Space Detection

### Current Limitation

The current system depends on a COCO annotation file containing the bounding boxes of parking spaces.

Therefore, the system already knows where the parking spaces are located before classification begins.

### Future Enhancement

A future version could automatically detect parking spaces directly from the parking-lot image.

The workflow could become:

Parking-Lot Image



Automatic Parking-Space Detection



Individual Parking Spaces



Occupancy Classification



Empty / Occupied

This would remove the requirement for a separate annotation file during normal application usage.

## 14.3 Deep-Learning-Based Classification

The current implementation uses a classical **HOG + SVM** pipeline.

A future version could use a deep-learning model for parking-space occupancy classification.

Possible approaches include:

- Convolutional Neural Networks (CNNs).
- Transfer learning.
- Modern object-detection architectures.
- Lightweight vision models.

A deep-learning approach could learn visual features directly from the training images rather than requiring manually designed HOG features.

The project documentation also identifies deep-learning approaches as a possible future improvement.



## 14.4 Real-Time Video Processing

### Current Limitation

The current implementation analyzes individual images.

### Future Enhancement

The system could be extended to process:

- CCTV streams.
- Webcam input.
- Recorded parking videos.
- IP camera feeds.

The future workflow could be:

Live Camera



Video Frames



Parking-Space Detection



Occupancy Classification



Real-Time Statistics



Live Display

This would make the system more suitable for practical parking-management applications.

## 14.5 Web-Based Dashboard

A web-based interface could be developed to provide a more user-friendly representation of parking information.

A future dashboard could display:

- Total parking spaces.
- Available spaces.
- Occupied spaces.

- Occupancy percentage.
- Annotated parking-lot image.
- Historical occupancy information.
- Model confidence information.

For example:

|                             |     |  |
|-----------------------------|-----|--|
| PARKING OCCUPANCY           |     |  |
| Total Spaces                | 100 |  |
| Occupied                    | 59  |  |
| Available                   | 41  |  |
| Occupancy                   | 59% |  |
| [Parking-Lot Visualization] |     |  |

This would transform the current command-line application into a more accessible system.

#### 14.6 Mobile Application Integration

A future version could provide a mobile application through which users can:

- View parking availability.
- Check occupancy before arriving.
- View parking-lot status.
- Locate available parking areas.

The computer-vision system could operate as the backend analysis engine while the mobile application provides the user interface.

#### 14.7 Historical Occupancy Tracking

The current application produces the occupancy statistics for an individual analysis.

A future version could store results over time.

For example:

| Time  | Occupancy |
|-------|-----------|
| 08:00 | 35%       |
| 10:00 | 52%       |
| 12:00 | 78%       |
| 14:00 | 64%       |
| 16:00 | 71%       |
| 18:00 | 85%       |

This historical information could be used to generate:

- Daily occupancy graphs.
- Weekly trends.
- Peak-hour analysis.
- Average occupancy.
- Parking-demand patterns.

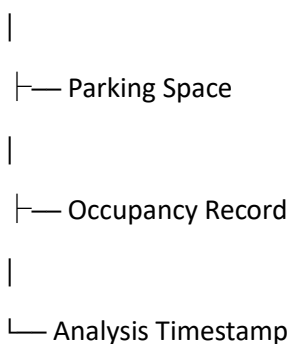
This enhancement would require persistent storage, such as a database.

### 14.8 Database Integration

The current system intentionally uses file-based storage rather than a database.

A future version could introduce a database containing entities such as:

Parking Lot



This would make it possible to maintain historical parking information and support multiple parking facilities.

An ER diagram would then become relevant to the system architecture.

### 14.9 Multi-Camera Support

Large parking facilities may contain multiple camera views.

A future system could process multiple camera feeds simultaneously:

Camera 1 —┐

Camera 2 —┐

Camera 3 —┴—► Occupancy Analysis —► Central Dashboard

Camera 4 —┐

This would allow the system to monitor different sections of a parking facility.

#### 14.10 Improved Robustness to Environmental Conditions

Real-world parking images can contain conditions that may differ from the training dataset.

Future development could improve performance under:

- Night-time conditions.
- Low illumination.
- Rain.
- Shadows.
- Glare.
- Different camera angles.
- Different parking-lot layouts.
- Partial vehicle visibility.

Additional training data and suitable image-augmentation techniques could be used to improve robustness.

#### 14.11 Confidence-Based Warning System

The current classifier provides prediction confidence.

A future system could use this confidence to identify uncertain predictions.

For example:

Confidence  $\geq 90\%$

↓

Normal Prediction

Confidence  $< 90\%$



Flag for Review

This could help prevent low-confidence predictions from being treated as completely reliable.

#### **14.12 Automatic Parking Availability Alerts**

A future system could generate notifications when parking availability reaches a particular condition.

For example:

Occupancy > 90%



Parking Lot Nearly Full



Generate Alert

Similarly, the system could notify users when a sufficient number of spaces become available.

This functionality would be particularly useful in large commercial or institutional parking facilities.

#### **14.13 Model Optimization**

The current HOG + SVM implementation can be further optimized for faster inference.

Possible improvements include:

- Feature-extraction optimization.
- Batch processing.
- Parallel processing.
- Model parameter tuning.
- Reducing unnecessary image operations.
- Hardware acceleration where applicable.

These improvements would become more important when processing large parking lots or real-time video.

#### **14.14 Automatic Model Retraining**

A future version could support periodic model retraining using newly collected parking-space images.

The workflow could be:

New Parking Images



Human/Automatic Validation



New Training Samples



Model Retraining



Model Evaluation



Updated Model

This could allow the classifier to adapt to new parking locations and changing environmental conditions.

#### 14.15 Improved Parking-Space Localization

The current system uses predefined bounding boxes.

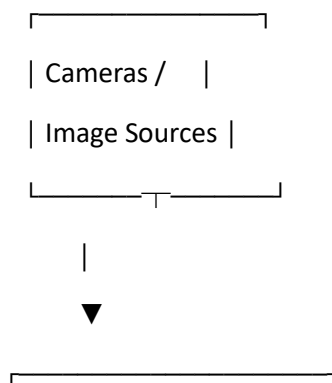
Future versions could investigate more advanced methods for locating parking spaces, including:

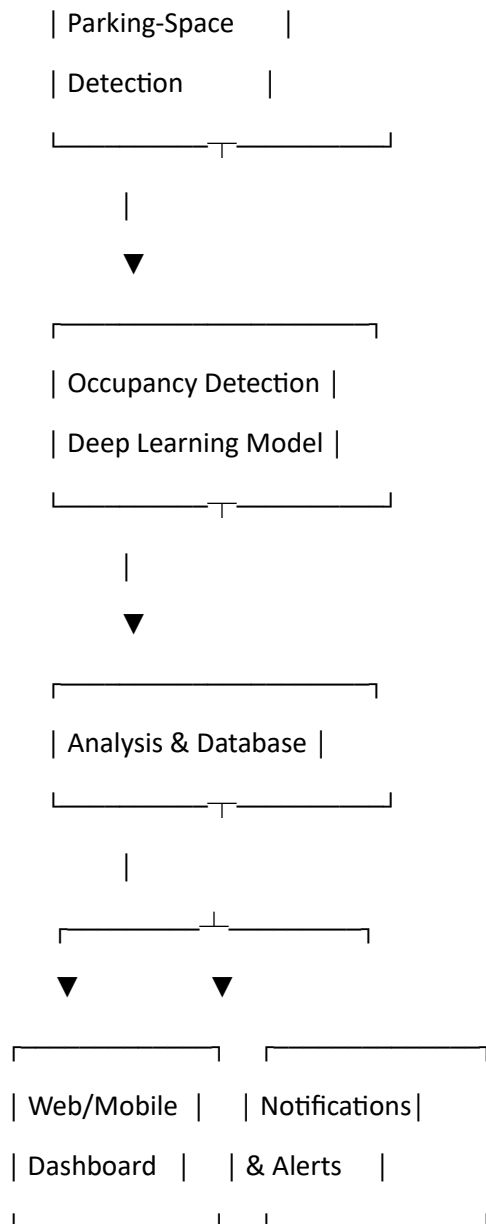
- Object detection.
- Segmentation.
- Perspective transformation.
- Automatic parking-slot mapping.

This would make the system less dependent on manually or externally supplied parking-space coordinates.

#### 14.16 Future System Architecture

With the proposed enhancements, the system could evolve from the current image-based CLI application into a complete real-time parking-management platform:





#### 14.17 Future Enhancement Summary

| Enhancement               | Expected Benefit                       |
|---------------------------|----------------------------------------|
| Automatic space detection | Removes dependency on annotation files |
| Deep-learning model       | Potentially improved feature learning  |
| Real-time video           | Continuous occupancy monitoring        |
| Web dashboard             | Easier visualization and management    |
| Mobile application        | User-facing parking availability       |
| Database                  | Historical and persistent records      |

| Enhancement               | Expected Benefit                        |
|---------------------------|-----------------------------------------|
| Multi-camera support      | Larger parking-area coverage            |
| Environmental robustness  | Better real-world performance           |
| Confidence-based warnings | Identification of uncertain predictions |
| Occupancy alerts          | Real-time parking notifications         |
| Model optimization        | Faster processing                       |
| Automatic retraining      | Adaptation to new data                  |

## 14.18 Conclusion

The current project provides a functional foundation for parking occupancy detection using computer vision and machine learning. Its architecture is modular enough to allow future replacement or extension of individual components.

The most significant next step would be to remove the dependency on manually supplied parking-space annotations by implementing **automatic parking-space detection**. After that, real-time video processing and a web/mobile interface could transform the current academic prototype into a more complete practical parking-management system.

These enhancements would progressively move the project from a **static image classification application** toward a **real-time intelligent parking monitoring system**.

## 15. References

The following references were used during the development and documentation of the **Computer Vision Based Parking Lot Occupancy Detection and Analysis System**.

### 15.1 Dataset References

1. **PKLot Dataset — Kaggle**

The parking-lot image dataset used as the source for the project dataset.

[PKLot Dataset on Kaggle](#)

### 15.2 Software and Library References

3. **OpenCV Documentation**

Used for image loading, image processing, cropping, and image annotation.

[OpenCV Documentation](#)

4. **Scikit-learn Documentation**

Used for SVM classification, model evaluation, and classification metrics.

[Scikit-learn Documentation](#)



5. **Scikit-image Documentation**

Used for Histogram of Oriented Gradients (HOG) feature extraction.

[Scikit-image Documentation](#)

6. **NumPy Documentation**

Used for numerical computation and array manipulation.

[NumPy Documentation](#)

7. **Pandas Documentation**

Used for dataset metadata processing and CSV report generation.

[Pandas Documentation](#)

8. **Pytest Documentation**

Used for automated testing of the project.

[Pytest Documentation](#)

9. **Joblib Documentation**

Used for serialization and storage of the trained machine-learning model.

[Joblib Documentation](#)

### 15.3 Computer Vision / Machine Learning References

10. **Dalal, N. and Triggs, B. — Histograms of Oriented Gradients for Human Detection**

N. Dalal and B. Triggs, *Histograms of Oriented Gradients for Human Detection*, IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), 2005.

This work provides the foundation for the HOG feature-descriptor approach used in the project.

11. **Cortes, C. and Vapnik, V. — Support-Vector Networks**

C. Cortes and V. Vapnik, *Support-Vector Networks*, Machine Learning, 20, 273–297, 1995.

This work provides the theoretical foundation for the Support Vector Machine classification method used in the project.

### 15.4 Project Documentation References

12. **Computer Vision Evaluated Project Guidelines — VIT**

Official project guidelines used to define the project requirements, functional and non-functional requirements, design documentation, implementation expectations, testing, GitHub requirements, and report structure.

13. **Project README — Computer Vision Based Parking Lot Occupancy Detection and Analysis System**

Project-specific documentation containing the dataset description, methodology, architecture, implementation workflow, evaluation results, testing procedure, limitations, and future enhancements.